

Vector spaces, matrices and norms

Vectors and vector spaces

- A vector v is an element of a vector space

* In this module, we will consider complex-valued vectors $x \in \mathbb{C}^n$, which have the form

$$x = [x_1, x_2, \dots, x_n]^T, \text{ where } x_i \in \mathbb{C}$$

- A vector space V is a set of vectors that is closed under addition and scalar multiplication

i.e. if $v_1, v_2 \in V$, then $av_1 + bv_2 \in V$ for any scalars a, b

(The scalars are often real/complex, but could also be chosen over some other field)

- A subspace of a vector space is a subset that obeys the rules of a vector space

- The inner product of two equal-length vectors yields a complex scalar, defined as

$$x^H y = \sum_{i=1}^n \bar{x}_i y_i$$

* $x^H y = \overline{y^H x}$ and $x^H x$ is real and non-negative ($x^H x = 0$ iff $x = 0$)

Matrices

- A matrix A is a linear operator that performs a linear transformation from one vector to another.

For a matrix $A: \mathbb{C}^n \rightarrow \mathbb{C}^m$ where $Ax = b$, the vector $x \in \mathbb{C}^n$ is mapped to the vector $b \in \mathbb{C}^m$

- An $m \times n$ matrix comes from the space $\mathbb{C}^{m \times n}$, so we often denote $A \in \mathbb{C}^{m \times n}$

- The product of two matrices $C = AB$ can be expressed using index notation

$$C_{ij} = \sum_k A_{ik} B_{kj} = A_{ik} B_{kj}$$

Treating a column vector as a $n \times 1$ matrix, we can compute the matrix-vector product $b = Ax$.

$$b_i = \sum_j A_{ij} x_j = x_1 \begin{bmatrix} \uparrow \\ a_{i1} \\ \downarrow \end{bmatrix} + x_2 \begin{bmatrix} \uparrow \\ a_{i2} \\ \downarrow \end{bmatrix} + \dots \leftarrow a_i \text{ are columns of } A$$

- The conjugate transpose / Hermitian transpose of a matrix is defined as

$$(A^H)_{ij} = \bar{A}_{ji}$$

Alternatively, using the transpose, $A^H = \bar{A}^T = \bar{A}^T$ (conjugate transpose $\equiv$ transpose for real-valued matrices)

- A matrix $M \in \mathbb{C}^{n \times n}$ is Hermitian if

$$M^H = M$$

- A Hermitian matrix $M \in \mathbb{C}^{n \times n}$ has the following properties:

(i) All eigenvalues are real, i.e. $\lambda_i \in \mathbb{R}$

If (λ, x) is an eigenpair of M , then $Mx = \lambda x$.

Pre-multiplying by x^H ,

$$x^H M x = \lambda x^H x$$

Taking the complex conjugate

$$(x^H M x)^H = x^H M x = \bar{\lambda} x^H x \quad (\text{use } M^H = M)$$

Comparing the above eqns, they can only hold when $\bar{\lambda} = \lambda$,

i.e. all the eigenvalues are real.

For Personal Use Only -bkwk2

(i) the eigenvectors are orthogonal, i.e. $\underline{x}_i^H \underline{x}_j = 0 \text{ if } i \neq j$.

consider two eigenvectors of $\underline{M}$, $(\lambda_1, \underline{x}_1), (\lambda_2, \underline{x}_2)$, i.e. $\underline{M}\underline{x}_1 = \lambda_1 \underline{x}_1$, $\underline{M}\underline{x}_2 = \lambda_2 \underline{x}_2$, where $\lambda_1 \neq \lambda_2$

premultiplying by $\underline{x}_2^H / \lambda_2^H$, $\underline{x}_2^H \underline{M} \underline{x}_1 = \lambda_1 \underline{x}_2^H \underline{x}_1$ $\underline{x}_2^H \underline{M} \underline{x}_2 = \lambda_2 \underline{x}_2^H \underline{x}_2$

Taking the complex conjugate for left eqn, $(\underline{x}_2^H \underline{M} \underline{x}_1)^H = \underline{x}_1^H \underline{M} \underline{x}_2 = \lambda_1 \underline{x}_1^H \underline{x}_2$ (use $\underline{M}^H = \underline{M}$)

Comparing the above eqns, since $\lambda_1 \neq \lambda_2$, they can only hold if $\underline{x}_1^H \underline{x}_2 = 0$

i.e. the eigenvectors are orthogonal.

- If the eigenvalues of the matrix $\underline{A} \in \mathbb{C}^{n \times n}$ are λ_i , then

(i) The eigenvalues of the inverse $\underline{A}^{-1}$ are $1/\lambda_i$

$$\underline{A} \underline{x}_i = \lambda_i \underline{x}_i \quad \rightarrow \quad \underline{A}^{-1} \underline{x}_i = \frac{1}{\lambda_i} \underline{x}_i$$

(ii) The eigenvalues of the transpose $\underline{A}^T$ are λ_i Note that $\det(\underline{A}^T) = \det(\underline{A})$

$$\det(\underline{A} - \lambda_i \underline{I}) = 0 \quad ; \quad 0 = \det(\underline{A}^T - \lambda_i \underline{I}) = \det((\underline{A} - \lambda_i \underline{I})^T) = \det(\underline{A} - \lambda_i \underline{I}) \rightarrow \lambda = \lambda_i$$

(iii) The eigenvalues of the conjugate transpose $\underline{A}^H$ are $\bar{\lambda}_i$. Note that $\det(\underline{A}^H) = \det(\underline{A}^T) = \det(\underline{A})$

$$\det(\underline{A} - \lambda_i \underline{I}) = 0 \quad ; \quad 0 = \det(\underline{A}^H - \lambda_i \underline{I}) = \det((\underline{A} - \bar{\lambda}_i \underline{I})^H) = \det(\underline{A} - \bar{\lambda}_i \underline{I}) = \det(\underline{A} - \lambda_i \underline{I}) \rightarrow \lambda = \bar{\lambda}_i$$

- For a Hermitian matrix $\underline{M} \in \mathbb{C}^{n \times n}$, it is positive definite / semipositive definite if

$$\underline{x}^H \underline{A} \underline{x} > 0 \quad \forall \underline{x} \in \mathbb{C}^n \setminus \underline{0} \quad \underline{x}^H \underline{A} \underline{x} \geq 0 \quad \forall \underline{x} \in \mathbb{C}^n \setminus \underline{0}$$

The eigenvalues of a Hermitian positive definite / semipositive definite matrix are strictly +ve / +ve.

$$\lambda_i > 0 \quad \lambda_i \geq 0$$

(say $(\lambda_i, \underline{x}_i)$ is an eigenvector of $\underline{M}$. $\underline{x}_i^H \underline{A} \underline{x}_i = \underline{x}_i^H (\lambda_i \underline{x}_i) = \lambda_i \underline{x}_i^H \underline{x}_i$, and $\underline{x}_i^H \underline{x}_i$ is real and positive)

+ Note that any Hermitian positive definite matrix is invertible, since $\lambda_i > 0 \rightarrow \det(\underline{M}) = \prod \lambda_i \neq 0$

- For a matrix $\underline{A} \in \mathbb{C}^{m \times n}$, the matrix $\underline{A}^H \underline{A}$ is semipositive definite, i.e. $\underline{x}^H \underline{A}^H \underline{A} \underline{x} \geq 0$.

$$0 \leq \|\underline{A} \underline{x}\|_2^2 = (\underline{A} \underline{x})^H (\underline{A} \underline{x}) = \underline{x}^H \underline{A}^H \underline{A} \underline{x}$$

so the eigenvalues of $\underline{A}^H \underline{A}$ are non-negative.
 if the eigenvalues of $\underline{A}$ are λ_i , then the eigenvalues of $\underline{A}^H \underline{A}$ are $\lambda_i \bar{\lambda}_i = |\lambda_i|^2$

- A matrix $\underline{Q} \in \mathbb{C}^{m \times m}$ is a unitary matrix if $\underline{Q}^H = \underline{Q}^{-1}$ i.e. $\underline{Q}^H \underline{Q} = \underline{Q} \underline{Q}^H = \underline{I}$.

$$\hookrightarrow |\det(\underline{Q})| = 1 \quad (\text{since } \det(\underline{Q}^H \underline{Q}) = \det(\underline{Q}^H) \det(\underline{Q}) = |\det(\underline{Q})|^2 = \det(\underline{I}) = 1)$$

$\hookrightarrow$ Transformation preserves the Euclidean norm of a vector $\|\underline{Q} \underline{x}\|_2 = \|\underline{x}\|_2$ and matrix $\|\underline{Q} \underline{A}\|_2 = \|\underline{A}\|_2$

$$(\|\underline{Q} \underline{x}\|_2^2 = (\underline{Q} \underline{x})^H (\underline{Q} \underline{x}) = \underline{x}^H \underline{Q}^H \underline{Q} \underline{x} = \underline{x}^H \underline{I} \underline{x} = \underline{x}^H \underline{x} = \|\underline{x}\|_2^2)$$

- The rank of a matrix $\underline{A} \in \mathbb{C}^{m \times n}$ is the no. of linearly independent rows or columns, and satisfies

$$\text{rank}(\underline{A}) \leq \min(m, n)$$

It is full rank if $\text{rank}(\underline{A}) = \min(m, n)$ and rank deficient if $\text{rank}(\underline{A}) < \min(m, n)$

- A matrix in which most entries are zero is a sparse matrix — the no. of non-zeroes on each row of a sparse matrix will be roughly the same and independent of the matrix size.

$\hookrightarrow$ The no. of entries in a dense matrix is $n \times n$

$\hookrightarrow$ The no. of entries in a sparse matrix is cn

For Personal Use Only -bkwk2

NORMS

- Denoting a norm of a vector x by $\|x\|$, a norm is a scalar that satisfies

$$\hookrightarrow \|x\| > 0 \text{ when } x \neq 0$$

$$\hookrightarrow \|x\| = 0 \text{ when } x = 0$$

$$\hookrightarrow \|kx\| = |k| \|x\|$$

$$\hookrightarrow \|x+y\| \leq \|x\| + \|y\|$$

* for orthogonal vectors x, y , i.e. $x^T y = 0$,
 $\|x \pm y\|^2 = \|x\|^2 \pm \|y\|^2$

Vector norms

- A particular family of norms are known as l_p norms, defined as

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \text{ where } |x| = \sqrt{\operatorname{Re}[x]^2 + \operatorname{Im}[x]^2}$$

$$\hookrightarrow l_1 \text{ norm: } \|x\|_1 = |x_1| + \dots + |x_n| = \sum_{i=1}^n |x_i| \quad [\text{taxi-cab norm}]$$

$$\hookrightarrow l_2 \text{ norm: } \|x\|_2 = \sqrt{|x_1|^2 + \dots + |x_n|^2} = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2} = (x^T x)^{1/2} \quad [\text{Euclidean norm}]$$

$$\hookrightarrow l_\infty \text{ norm: } \|x\|_\infty = \max_i |x_i| \quad [\text{max. norm}]$$

- We can define norms that involve a matrix A if A is positive definite.

$$\|x\|_A = \sqrt{x^T A x} \quad [\text{energy norm}]$$

if $A = I$, we have the l_2 norm

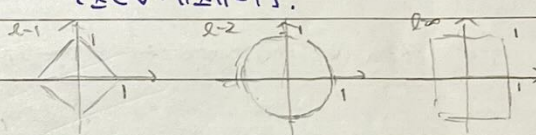
(A being positive definite implies that the energy $x^T A x$ is non-negative and only zero if $x=0$)

- It is useful to refer to the unit ball/sphere for a given norm.

$$\{x \in V : \|x\| \leq 1\}$$

$$\{x \in V : \|x\| = 1\}$$

The "shape" depends on the chosen norm



Matrix norms induced by vector l_p norms

- The norm of a matrix A is defined as

$$\|A\| = \max_{x \in \mathbb{C}^n, \|x\|=1} \|Ax\|$$

or

$$\|A\| = \max_{\substack{x \in \mathbb{C}^n \\ \|x\|=1}} \|Ax\|$$

i.e. the norm measures the "max out" by which the matrix A can rescale a vector x

- We can express the defn of the operator norm as an inequality

$$\|Ax\| \leq \|A\| \|x\| \quad \forall x$$

It follows from this that (i) $\|AB\| \leq \|A\| \|B\|$ and (ii) $\|A+B\| \leq \|A\| + \|B\|$

$$(i) \quad \|A(Bx)\| \leq \|A\| \|Bx\| \leq \|A\| (\|B\| \|x\|) \rightarrow \frac{\|ABx\|}{\|x\|} \leq \|A\| \|B\| \quad \forall x$$

$$\text{since } \|AB\| = \max_{x \in \mathbb{C}^n, \|x\|=1} \frac{\|ABx\|}{\|x\|}, \quad \|AB\| = \frac{\|ABx\|}{\|x\|} \text{ for a particular } x \rightarrow \|AB\| \leq \|A\| \|B\|$$

$$(ii) \quad \|(A+B)x\| = \|Ax+Bx\| \leq \|Ax\| + \|Bx\| \leq \|A\| \|x\| + \|B\| \|x\| \rightarrow \frac{\|(A+B)x\|}{\|x\|} \leq \|A\| + \|B\|$$

$$\text{since } \|A+B\| = \max_{x \in \mathbb{C}^n, \|x\|=1} \frac{\|(A+B)x\|}{\|x\|}, \quad \|A+B\| = \frac{\|(A+B)x\|}{\|x\|} \text{ for a particular } x \rightarrow \|A+B\| \leq \|A\| + \|B\|$$

- To quantify the "size" of the original vector x and the transformed vector Ax , we need to choose a norm

$$l_1 \text{ norm: } \|A\|_1 = \max_{x \in \mathbb{C}^n, \|x\|=1} \|Ax\|_1 = \max_j \sum_{i=1}^n |a_{ij}| \quad \leftarrow l_1 \text{ norm of the column of } A \text{ w/ the max. } l_1 \text{ norm}$$

$$l_2 \text{ norm: } \|A\|_2 = \max_{x \in \mathbb{C}^n, \|x\|=1} \|Ax\|_2 = \sqrt{\lambda_{\max}(A^H A)} \quad \leftarrow \text{sqrt of the largest eigenvalue of } A^H A \text{ or largest singular value of } A$$

$$l_\infty \text{ norm: } \|A\|_\infty = \max_{x \in \mathbb{C}^n, \|x\|=1} \|Ax\|_\infty = \max_i \sum_{j=1}^n |a_{ij}| \quad \leftarrow l_1 \text{ norm of the row of } A \text{ w/ the max. } l_1 \text{ norm}$$

* If A is Hermitian, the eigenvalues of $A^H A = A^2$ are the square of eigenvalues of A , so $\|A\|_2 = \sqrt{\lambda_{\max}(A)}$

For Personal Use Only -bkwk2

- For the ℓ_2 norm of a matrix, $\|A\|_2$,

$$\|A\|_2^2 = \max_{\|x\|_2=1} \|Ax\|_2^2 = \max_{x^H x = 1} \frac{(Ax)^H (Ax)}{x^H x} = \max_{x^H x = 1} \frac{x^H A^H A x}{x^H x} = \frac{x^H \lambda_{\max}(A^H A) x}{x^H x} = \lambda_{\max}(A^H A)$$

$$\therefore \|A\|_2 = \sqrt{\lambda_{\max}(A^H A)}$$

→ Note $\|A^H\|_2 = \sqrt{\lambda_{\max}(A A^H)}$ since $\lambda(A A^H) = \lambda(A^H A)$, $\rightarrow \lambda_{\max}(A^H A) = \lambda_{\min}(A A^H)$

- For the ℓ_1 and ℓ_∞ norm of a matrix, $\|A\|_1, \|A\|_\infty$,

Recall that the operator norm $\|A\|$ is the min. value of C that satisfies $\|Ax\| \leq C\|x\|$, and set $\|x\|=1$

(i) ℓ_1 norm: $\|A\|_1 = \left\| \sum_{j=1}^n x_j a_j \right\|_1 \leq \sum_{j=1}^n \|x_j a_j\|_1 = \sum_{j=1}^n |x_j| \|a_j\|_1 \leq \max_{j \in [1, n]} \|a_j\|_1$ (since $\|x\|_1=1, \sum |x_j|=1$)

choose a vector x s.t. $x_i = \begin{cases} 1 & \text{if } i=j \\ 0 & \text{o.w.} \end{cases}$. then $\|Ax\|_1 = \|a_j\|_1 \rightarrow \|A\|_1 = \max_{j \in [1, n]} \|a_j\|_1$

(ii) ℓ_∞ norm: $\|A\|_\infty = \max_{i \in [1, m]} \left| \sum_{j=1}^n x_j a_{ij} \right| \leq \max_{i \in [1, m]} \sum_{j=1}^n |x_j| |a_{ij}| \leq \max_{i \in [1, m]} \sum_{j=1}^n |a_{ij}|$ (since $\|x\|_\infty=1, x_i \leq 1$)

choose a vector x s.t. $x_i = \text{sign}(a_{ij})$, then $\|Ax\|_\infty = \sum_{j=1}^n |a_{ij}| \rightarrow \|A\|_\infty = \max_{i \in [1, m]} \sum_{j=1}^n |a_{ij}|$

→ Note that the vector and matrix ℓ_2 norms are invariant under rotation, i.e. for a unitary matrix Q ,

$$\|Qx\|_2 = \|x\|_2$$

$$\|QA\|_2 = \|A\|_2$$

Frobenius norm

- The Frobenius norm of a matrix A is defined as

$$\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2}$$

It can be shown that the Frobenius norm is also given by the following.

$$\|A\|_F = \sqrt{\text{trace}(A^H A)} = \sqrt{\sum_{i=1}^{\min(m,n)} \sigma_i^2(A)}$$

→ Note that the Frobenius norm of a matrix is invariant under rotation, i.e. for a unitary matrix Q ,

$$\|QA\|_F = \|A\|_F$$

Equivalence of norms

- Choosing a norm depends on what we want to measure

→ e.g. ℓ_1 norm for probabilities, ℓ_2 norm for distances

- On all the vector spaces that we consider, all norms are equivalent, i.e. $\exists$ constants $c_1 > 0, c_2 > 0$ s.t.

$$c_1 \|x\|_x \leq \|x\|_y \leq c_2 \|x\|_x \quad \forall x \in V$$

Stability and condition number

condition number.

- The condition no. of an invertible matrix A is defined by

$$K(A) = \|A\| \|A^{-1}\|$$

Note that $K(A) \geq 1$ since $K(A) = \|A\| \|A^{-1}\| \geq \|AA^{-1}\| = \|I\| = 1$ - For the ℓ_2 norm, $\|A\|_2 = \sqrt{\lambda_{\max}(A^H A)}$ and $\|A^{-1}\|_2 = \sqrt{\lambda_{\min}(A^H A)^{-1}}$, so

$$K_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sqrt{\lambda_{\max}(A^H A)}}{\sqrt{\lambda_{\min}(A^H A)}}$$

If A is Hermitian, then $K_2(A) = \frac{|\lambda_{\max}(A)|}{|\lambda_{\min}(A)|}$ • For small K , the shape dilates (stretch equally in all dir.)For large K , the shape stretches in a particular dir. (in the limit collapses into a line)

Error bounds and stability.

- Consider the problem where we have an error in making the observation b

$$A(x + \delta x) = b + \delta b$$

where δb is the error in the observation and δx is the consequent error in the soln.

$$Ax = b : \|A\| \|x\| \geq \|Ax\| = \|b\| \rightarrow \frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|} \quad [1]$$

$$A\delta x = \delta b : \delta x = A^{-1} \delta b \rightarrow \|\delta x\| = \|A^{-1} \delta b\| \leq \|A^{-1}\| \|\delta b\| \quad [2]$$

$$\text{Combining [1], [2],} \quad \frac{\|\delta x\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|} = K(A) \frac{\|\delta b\|}{\|b\|}$$

i.e. for large condition no. $K(A)$, we can expect small errors in b to cause large errors in x .- Consider the problem where we have an error in the matrix A (e.g. floating pt. error in LU decomposition)

$$(A + \delta A)(x + \delta x) = b$$

where δA is the error in the matrix and δx is the consequent error in the soln.

$$(A + \delta A)(x + \delta x) = Ax + A\delta x + \delta A(x + \delta x) = b \rightarrow \delta x = -A^{-1} \delta A(x + \delta x)$$

$$\therefore \|\delta x\| = \|A^{-1} \delta A(x + \delta x)\| \leq \|A^{-1}\| \|\delta A\| \|x + \delta x\| \rightarrow \frac{\|\delta x\|}{\|x + \delta x\|} \leq \|A^{-1}\| \|\delta A\| \frac{\|x\|}{\|b\|} = K(A) \frac{\|\delta A\|}{\|A\|}$$

i.e. for large condition no. $K(A)$, we can expect small errors in A to cause large errors in x .- A matrix w/ a large condition no. $K(A)$ is said to be ill-conditioned ($\neq$ small determinant)• Conditioning shouldn't be confused w/ the determinant (i.e. large condition no. $\neq$ small determinant).

$$\hookrightarrow A = \begin{bmatrix} 1 & -1 & \dots & -1 \\ 0 & 1 & \dots & -1 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} \quad \det(A) = 1, \quad K_2(A) = n 2^{n-1}$$

$$\hookrightarrow D = \text{diag}(0.1, 0.1, \dots, 0.1) \quad \det(D) = 10^{-n}, \quad K_2(D) = 1$$

Interpolation and least squares methods

Polynomial interpolation.

- If we have n data pts in a 2D space (x, y) , we can usually fit a polynomial w/ n coeffs

e.g. given n data pts $f = [f_0(x_0, y_0) \dots f_n(x_n, y_n)]^T$, we can interpolate the data pts w/ a

polynomial of the form $f(x, y) = c_0 + c_1x + c_2y + c_3xy + c_4x^2 + c_5y^2$

For each data pt $f(x_i, y_i)$, we have an eqn, so we can write our system of eqns in matrix form

$$\underbrace{\begin{bmatrix} 1 & x_0 & y_0 & x_0y_0 & x_0^2 & y_0^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & y_n & x_ny_n & x_n^2 & y_n^2 \end{bmatrix}}_{\text{Vandermonde matrix } A} \underbrace{\begin{bmatrix} c_0 \\ \vdots \\ c_5 \end{bmatrix}}_c = \underbrace{\begin{bmatrix} f_0 \\ \vdots \\ f_n \end{bmatrix}}_f$$

- In general, there are a few issues w/ solving for c , i.e. finding the interpolating polynomial:

↳ We can only solve the matrix eqn when A is full rank, i.e. data pts are not colinear

↳ The matrix A is notoriously ill-conditioned, and the condition no. $\kappa(A)$ grows w/ polynomial degree n .

↳ If we choose equally spaced interpolation pts, we get the Runge effect - large amplitude oscillations near the ends of the domain, which become more severe at high polynomial degrees.

Using a different set of basis functions

- So far, we have considered the monomial basis $\{1, x, \dots, x^{n-1}\}$, so we have polynomials of the form

$$f = c_{n-1}x^{n-1} + c_{n-2}x^{n-2} + \dots + c_1x + c_0$$

where we would solve for the coefficients c by solving the matrix eqn.

$$\begin{bmatrix} 1 & x_0 & \dots & x_0^{n-2} & x_0^{n-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & x_{n-1} & \dots & x_{n-1}^{n-2} & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-2} \\ c_{n-1} \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_{n-2} \\ f_{n-1} \end{bmatrix}$$

- Instead, we could consider the Legendre polynomial basis $\{P_0(x), P_1(x), \dots, P_n(x)\}$, where

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x), \quad \text{and } P_0 = 1, P_1 = x$$

Note that the Legendre polynomials form an orthogonal basis on the interval $[-1, 1]$, i.e.

$$\langle P_m, P_n \rangle = \int_{-1}^1 P_m(x)P_n(x)dx = 0 \quad \text{if } m \neq n$$

Therefore, we have polynomials of the form

$$f = \alpha_{n-1}P_{n-1}(x) + \alpha_{n-2}P_{n-2}(x) + \dots + \alpha_1P_1(x) + \alpha_0P_0(x)$$

where we would solve for the coefficients α by solving the matrix eqn.

$$\begin{bmatrix} P_0(x_0) & P_1(x_0) & \dots & P_{n-2}(x_0) & P_{n-1}(x_0) \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ P_0(x_{n-1}) & P_1(x_{n-1}) & \dots & P_{n-2}(x_{n-1}) & P_{n-1}(x_{n-1}) \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{n-2} \\ \alpha_{n-1} \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_{n-2} \\ f_{n-1} \end{bmatrix}$$

- Using the Legendre polynomials instead of monomials as the basis significantly reduces the condition no. $\kappa(A)$ of the Vandermonde matrix A .

For Personal Use Only -bkwk2

Interpolation w/ non-equispaced points.

- The Lebesgue const. $\Lambda(T)$ bounds the interpolation error. For a function f , where p_n is the best approx of f amongst all polynomials of degree n and p_T is the degree n polynomial interpolant of f ,

$$\|f - p_T\| \leq (\Lambda(T) + 1) \|f - p_n\|$$

Note that the Lebesgue const. $\Lambda(T)$ depends on the choice of interpolation pts T .

- For a good polynomial interpolant p_T , we want the Lebesgue const. $\Lambda(T)$ to be as small as possible.
- By using non-equispaced sampling pts - roots of orthogonal polynomials in particular (they tend to cluster close to the ends of the interval), we can mitigate the Runge effect (i.e. small $\Lambda(T)$)
- The Lebesgue const. $\Lambda(T)$ grows exponentially in n for equispaced pts but only logarithmically in n for Chebyshev pts (roots of Chebyshev polynomials, defined as $T_n(x) = \cos(n \arccos(x))$, $x \in [-1, 1]$).

Least squares method for data fitting of over-determined systems.

- To find a sol'n to the matrix eqn $Ax = b$, the vector b must lie in the column space of A .
If A is a $m \times n$ matrix w/ $m > n$ (stubby matrix), in general b does not lie in the column space of A .
- For an approx sol'n $\hat{x}$, we can define the residual vector r as

$$r = A\hat{x} - b$$

We want to choose $\hat{x}$ s.t. the norm of the residual vector $\|r\|$ is min. for some norm, i.e.

$$\hat{x}^* = \arg \min_{x \in \mathbb{C}^n} \|r(x)\| = \arg \min_{x \in \mathbb{C}^n} \|A\hat{x} - b\|$$

- If we use the ℓ_2 norm, we seek $\hat{x}^* = \arg \min_{x \in \mathbb{C}^n} \|A\hat{x} - b\|_2$. Let $r(\hat{x}) = \|A\hat{x} - b\|_2^2$, we have

$$r(\hat{x}) = \|A\hat{x} - b\|_2^2 = \sum_{i=1}^m \left| \sum_{j=1}^n (a_{ij}\hat{x}_j) - b_i \right|^2 \rightarrow \text{least squares method}$$

Expanding and minimising $r(\hat{x})$.

$$r(\hat{x}) = \|A\hat{x} - b\|_2^2 = (A\hat{x} - b)^H (A\hat{x} - b) = \hat{x}^H A^H A \hat{x} - \hat{x}^H A^H b - b^H A \hat{x} + b^H b$$

$$\therefore \frac{dr(\hat{x})}{d\hat{x}} = 2A^H A \hat{x} - 2A^H b = 0 \rightarrow A^H A \hat{x}^* = A^H b$$

Rearranging the least squares problem, we have

$$\hat{x}^* = (A^H A)^{-1} A^H b = A^+ b$$

where $A^+ = (A^H A)^{-1} A^H$ is the pseudoinverse / penrose-Moore inverse.

* The inverse $(A^H A)^{-1}$ can be computed when A is full rank

- To solve the least squares problem $A^H A \hat{x}^* = A^H b$, if A is full rank, we can apply LU factorisation or Cholesky factorisation to $A^H A$, but $A^H A$ is notoriously ill-conditioned

- We can use more specialised methods like QR factorisation or SVD to mitigate the

ill-conditioning of $A^H A$ (QR may have accuracy/stability problems and SVD is expensive)

* In practice, do not write your own least-squares solver $\rightarrow$ use specialised functions/libraries

Iterative methods for linear systems

Direct and iterative methods.

- To solve a system of the form $\underline{A}\underline{x} = \underline{b}$, solns can be categorised as direct or iterative.

① Direct methods.

- A direct method computes a soln to $\underline{A}\underline{x} = \underline{b}$ in a known/predictable no. of operations. w/o round-off errors, the soln is exact. (eg LU factorisation)
- In general, direct methods cannot handle singular eqns
- For reasonably conditioned systems, direct solvers can be robust.
- Direct solvers do not scale well on parallel computers as substitution steps are serial.

② Iterative methods.

- An iterative method seeks an approx soln via a series of iterations, and is usually terminated when the error/residual falls below a prescribed threshold.
- Some iterative methods can solve singular eqns.
- Iterations can be terminated early if high accuracy is not req.
- Iterative methods use less memory and are suited to large parallel computers.

Power iteration for eigenvector w/ largest eigenvalue

- We can use power iteration to find the eigenvector associated w/ the largest absolute eigenvalue.
- A vector $\underline{x} \in \mathbb{C}^n$ can be expressed in terms of the n eigenvectors of an $n \times n$ matrix.

$$\underline{x} = \sum_{i=1}^n \alpha_i \underline{u}_i \quad \text{where } \alpha_i \in \mathbb{R}.$$

If we multiply $\underline{x}$ repeatedly by $\underline{A}$,

$$\underline{A}^k \underline{x} = \sum_{i=1}^n \alpha_i \lambda_i^k \underline{u}_i$$

$\therefore$ if the largest eigenvalue is distinct, the resulting vector will be aligned w/ the eigenvector of the largest eigenvalue (given $\underline{x}$ has a component in the dir. of the eigenvector)

- similar methods inc. the inverse power method $\underline{A}^{-1} \underline{x} = \frac{1}{\lambda} \underline{x} \rightarrow$ eigenvector w/ smallest eigenvalue
- and the shifted inverse power method $(\underline{A} - \alpha \underline{I})^{-1} \underline{x} = \frac{1}{\lambda - \alpha} \underline{x} \rightarrow$ eigenvector w/ eigenvalue closest to α .

* These methods can converge slowly

For Personal Use Only -bkwk2

Rayleigh quotient for eigenvalues

- we can use the Rayleigh quotient $R(A, x)$ to find the eigenvalue λ^1 corresponding to the given estimate of an eigenvector x .

- Let x be an estimate of an eigenvector of the matrix A , so $Ax \approx \lambda x$.

To find an estimate of the corresponding eigenvalue λ^1 , we consider the minimisation problem in ℓ_2 norm,

$$\lambda^1 = \min_{\lambda \in \mathbb{C}} \|Ax - \lambda x\|_2$$

This is minimised when λ^1 is equal to the Rayleigh quotient $R(A, x)$.

$$\lambda^1 = R(A, x) = \frac{x^H A x}{x^H x}$$

- to prove this, consider any $\mu \in \mathbb{C}$,

$$\begin{aligned} \|Ax - \mu x\|_2^2 &= x^H A^H A x - \mu x^H A x - \bar{\mu} x^H A x + |\mu|^2 x^H x = \|Ax\|_2^2 + \|x\|_2^2 (|\mu|^2 - \mu \bar{\lambda}^1 - \bar{\mu} \lambda^1) \\ &= \|Ax\|_2^2 + \|x\|_2^2 ((\mu - \bar{\lambda}^1)(\bar{\mu} - \lambda^1) - |\lambda^1|^2) \geq \|Ax\|_2^2 - |\lambda^1|^2 \|x\|_2^2 = \|Ax - \lambda^1 x\|_2^2 \end{aligned}$$

- The Rayleigh quotient $R(A, x)$ is often defined for Hermitian matrices only, where it is real-valued.

It also has other properties, inc. second-order accuracy for estimating eigenvalues

eigenvector error $O(\epsilon)$, eigenvalue error $O(\epsilon^2)$

Stationary methods for $Ax=b$

- consider decomposing the matrix operator s.t. $A = N - P$. The matrix eqn. $Ax=b$ becomes,

$$(N - P)x = b \quad \rightarrow \quad Nx = b + Px$$

We can compute an approx soln x_{k+1}

$$Nx_{k+1} = b + Px_k$$

where x_k is the most recent known estimate of the soln

- We keep iterating for the new estimate x_{k+1} and hopefully this converges to the exact soln.

- The key is to split A s.t. eqns of the form $Nx = f$ are easy (cheap) to solve, examples inc.

↳ Richardson: $N = I$

↳ Jacobi: $N = \text{diag}(A)$

↳ Gauss-Seidel: $N = L(\underline{A})$ (lower triangular part of A , inc. the diagonal).

- defining the error at the k th iterate, as $e_k = x_{\text{exact}} - x_k$, we have,

$$N(x_{\text{exact}} - e_{k+1}) = (N - P)x_{\text{exact}} + P(x_{\text{exact}} - e_k) \rightarrow Ne_{k+1} = Pe_k$$

$$\therefore e_{k+1} = N^{-1}P e_k$$

Expressing the initial error vector e_0 as a linear combination of the eigenvectors of $N^{-1}P$,

$$e_0 = c_1 u_1 + \dots + c_n u_n$$

The error e_k is therefore

$$e_k = (N^{-1}P)^k e_0 = c_1 \lambda^k u_1 + \dots + c_n \lambda^k u_n$$

- For convergence, we req. $e_k \rightarrow 0$ as k increases, which is true if the absolute value of every eigenvalue

of $N^{-1}P$ is less than one — i.e. the spectral radius of $N^{-1}P$ is less than 1, $\rho(N^{-1}P) < 1$.

* The smaller $\rho < 1$, the faster the convergence.

For Personal Use Only -bkwk2

Conjugate gradient (CG) method

- The CG method is a Krylov subspace method for solving $Ax=b$, where A is Hermitian, positive definite, and is more powerful than the stationary methods outlined above.

- The CG method is often presented as a direct method, but is applied in practice usually as an iterative method.

① CG method as a direct method.

- consider $Ax=b$, where A is a non Hermitian positive-definite matrix.

consider that we have a set P of n non-zero vectors $P = \{p_0, \dots, p_{n-1}\}$ that are A -conjugate, i.e.,

$$P_i^H A P_j = 0 \quad \text{if } i \neq j.$$

Note that since A is positive definite, $P_i^H A P_i > 0$

- Using P as a basis for the soln, $x = \sum_{i=0}^{n-1} \alpha_i P_i$, so we have

$$b = Ax = \sum_{i=0}^{n-1} \alpha_i A P_i$$

Pre-multiplying by P_j^H

$$P_j^H b = P_j^H A x = \sum_{i=0}^{n-1} \alpha_i P_j^H A P_i = \alpha_j P_j^H A P_j$$

We can therefore solve for all α_i and get $x = \sum_{i=0}^{n-1} \alpha_i P_i$, where α_i is given by

$$\alpha_j = \frac{P_j^H b}{P_j^H A P_j}$$

- To generate the A -conjugate set P , we can simply pick n linearly independent vectors and apply the Gram-Schmidt process to build P , but this is not practical for large problems ← build + store P too expensive

② CG method as an iterative method.

- The CG algorithm involves choosing elements of P using a short difference eqn. Moreover, we may not even need all of P - we can get an accurate soln w/ a few elements of P .

- The CG algorithm only involves matrix-vector products and dot products, and only req. storing a few work vectors (low space complexity)

- When solving $Ax=b$ where A is a $n \times n$ matrix;

↳ The error $e_k = x_{\text{exact}} - x_k$ is monotone in the A -norm ($\|e_k\|_A = \sqrt{e_k^H A e_k}$), i.e., $\|e_{k+1}\|_A \leq \|e_k\|_A$ and converges to 0, i.e. $\|e_k\|_A$ for some $k \leq n$. (in exact arithmetic)

↳ The no. of iterations req. to solve x exactly is equal to the no. of distinct eigenvalues of A (so $\leq n$)

↳ The rate of convergence is affected by the condition no. $K_2(A)$,

$$\frac{\|e_k\|_A}{\|e_0\|_A} \leq 2 \left(\frac{\sqrt{K_2(A)} - 1}{\sqrt{K_2(A)} + 1} \right)^k$$

Note when $K_2(A)$ is close to 1, e_k converges to 0 quickly.

- If the condition no. $K_2(A)$ is too large, the CG method may converge too slowly for practical use

→ we can pre-condition the problem, where we consider the transformed system

left preconditioning. → $P^H A x = P^H b$

If $P^H A$ is much better than A → CG method converges faster

+ In practice, it can be a balancing act - P^H must be cheap to apply but close enough to A^H to be effective.

For Personal Use Only -bkwk2

Singular value decomposition (SVD)

Matrix diagonalisation.

- We can diagonalise any Hermitian matrix $M \in \mathbb{C}^{n \times n}$

$$M = Q \Lambda Q^H$$

where the columns of Q are the ℓ^2 normalised eigenvectors of M (which are orthogonal $\rightarrow Q$ is unitary).

and Λ is the diagonal matrix of the eigenvalues of M (which are real).

- We can interpret the diagonalisation as a sum of rank 1 matrices.

$$M = \sum_i \lambda_i u_i u_i^H$$

where (λ_i, u_i) is the i th eigenpair of M .

- Note that the eigenvectors are not unique $\rightarrow$ sign ambiguity $M = Q \Lambda Q^H = (-Q) \Lambda (-Q)^H$

- Matrix diagonalisation is v. helpful as we see that in a particular basis, the action of a matrix on a vector is easy to interpret, and matrix-matrix operations become trivial

- However, matrix diagonalisation has a few major drawbacks

$\hookrightarrow$ only valid for square matrices

$\hookrightarrow$ defective matrices cannot be diagonalised

$\hookrightarrow$ The matrix Q is guaranteed unitary only for Hermitian matrices M .

Singular value decomposition (SVD)

- The SVD of a $m \times n$ matrix A is

$$A = U \Sigma V^H$$

where $U \in \mathbb{C}^{m \times m}$ is a unitary matrix, $V \in \mathbb{C}^{n \times n}$ is a unitary matrix

$\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix, w/ diagonal entries σ_i (singular values) sorted st. $\sigma_1 \geq \dots \geq \sigma_p \geq 0$

$p = \min(m, n)$

- consider $S_R = A^H A$ (which is Hermitian)

$$S_R = A^H A = (U \Sigma V^H)^H (U \Sigma V^H) = V \Sigma^H U^H U \Sigma V^H = V \Sigma^H \Sigma V^H$$

$\rightarrow$ The columns of V are the normalised eigenvectors of $A^H A$

The diagonal entries of $\Sigma^H \Sigma = \sigma_1^2, \sigma_2^2, \dots, \sigma_p^2$ are the eigenvalues of $A^H A$

- consider $S_L = A A^H$ (which is Hermitian)

$$S_L = A A^H = (U \Sigma V^H) (U \Sigma V^H)^H = U \Sigma V^H V \Sigma^H U^H = U \Sigma \Sigma^H U^H$$

same eigenvalues

$\rightarrow$ The columns of U are the normalised eigenvectors of $A A^H$

The diagonal entries of $\Sigma \Sigma^H = \sigma_1^2, \sigma_2^2, \dots, \sigma_p^2$ are the eigenvalues of $A A^H$

- To determine the sign of the eigenvectors, postmultiply both sides of $A = U \Sigma V^H$ by U , so

$$A U = U \Sigma \quad \Leftrightarrow \quad A u_i = \sigma_i u_i$$

where u_i is the i th column of U and u_i is the i th column of U

Every matrix has a SVD, which is unique up to the signs of the eigenvectors in U and V .

* The SVD of a Hermitian matrix is the usual matrix diagonalisation.

For Personal Use Only -bkwk2

The reduced SVD.

- For a $m \times n$ matrix $w/ m > n$ (skinny matrix), the SVD can be reduced by removing redundant entries

$$\begin{bmatrix} A \\ m \times n \end{bmatrix} = \begin{bmatrix} U \\ m \times n \end{bmatrix} \begin{bmatrix} \Sigma \\ m \times n \\ 0 \end{bmatrix} \begin{bmatrix} V^T \\ n \times n \end{bmatrix} \rightarrow \begin{bmatrix} A \\ m \times n \end{bmatrix} = \begin{bmatrix} U \\ m \times n \end{bmatrix} \begin{bmatrix} \Sigma \\ m \times n \end{bmatrix} \begin{bmatrix} V^T \\ n \times n \end{bmatrix}$$

no contribution

The terms below row n in Σ are always zero $\rightarrow$ last $m-n$ columns of U make no contribution.

- For a $m \times n$ matrix $w/ m < n$ (fat matrix), the SVD can be reduced by removing redundant entries

$$\begin{bmatrix} A \\ m \times n \end{bmatrix} = \begin{bmatrix} U \\ m \times n \end{bmatrix} \begin{bmatrix} \Sigma \\ m \times n \\ 0 \end{bmatrix} \begin{bmatrix} V^T \\ n \times n \end{bmatrix} \rightarrow \begin{bmatrix} A \\ m \times n \end{bmatrix} = \begin{bmatrix} U \\ m \times n \end{bmatrix} \begin{bmatrix} \Sigma \\ m \times n \end{bmatrix} \begin{bmatrix} V^T \\ n \times n \end{bmatrix}$$

no contribution

The terms right of column n in Σ are always zero $\rightarrow$ last $n-m$ rows of V^T make no contribution

Low rank approximations

- If we expand the SVD $A = U \Sigma V^T$, we get a sum of rank-1 matrices.

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T$$

where r is the rank of A (no. of non-zero singular values),

and u_i is the i th column of U , v_i is the i th column of V .

- Noting that σ_i are sorted in descending order, a rank k approx of A is given by

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T, \quad k < r$$

A_k is a better approx of A than any other matrix of rank k or less, B_k .

$$(i) \|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2} \leq \|A - B_k\|_F \quad \text{or} \quad (ii) \|A - A_k\|_2 = \sigma_{k+1} \leq \|A - B_k\|_2$$

$$\rightarrow \text{proof of (ii): } A - A_k = \sum_{i=k+1}^r \sigma_i u_i v_i^T - \sum_{i=1}^k \sigma_i u_i v_i^T = \sum_{i=k+1}^r \sigma_i u_i v_i^T$$

$$\therefore \|A - A_k\|_2 = \left\| \sum_{i=k+1}^r \sigma_i u_i v_i^T \right\|_2 = \sigma_{k+1}$$

2-norm of a matrix is equal to the largest singular value

As B_k has rank k , there exists a vector $w = \alpha_1 v_1 + \dots + \alpha_k v_k$ that lies in the null space of B_k .

$$\text{Denoting } \Sigma_{k+1} \text{ as the first } k+1 \text{ entries of } \Sigma \text{ and } w \text{ as } w = \begin{bmatrix} v_1 \\ \vdots \\ v_{k+1} \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_{k+1} \end{bmatrix} = V_{k+1} \alpha,$$

For convenience, normalise w s.t. $\|w\|_2 = 1$ (so $|\alpha_i| < 1$)

$$\begin{aligned} \frac{1}{\|w\|_2} \|A - B_k\|_2^2 &\geq \|(A - B_k)w\|_2^2 = \|A w - B_k w\|_2^2 = \|U \Sigma V^T V_{k+1} \alpha\|_2^2 = \|\Sigma_{k+1} \alpha\|_2^2 \\ &= \alpha_1^2 \sigma_1^2 + \dots + \alpha_{k+1}^2 \sigma_{k+1}^2 \geq \sigma_{k+1}^2 = \|A - A_k\|_2^2 \rightarrow \|A - B_k\|_2 \geq \|A - A_k\|_2 \end{aligned}$$

by construction of w

Effective rank

- When taking measurements, noise is often unavoidable $\rightarrow$ hard to detect linear dependencies.

$$\rightarrow \text{eg. } A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 0 & 1 \end{bmatrix} \quad A_{\text{noisy}} = \begin{bmatrix} 1.0000022 & 1.0000006 & 1.0000014 \\ 2.0000018 & 2.0000015 & 2.0000011 \\ 1.0000008 & 2.1924 \times 10^{-6} & 1.0000012 \end{bmatrix} \quad (\text{noise} \in [0, 10^{-6}])$$

rank 2 rank 3

SVD on A_{noisy} gives $\sigma_1 = 4.048$, $\sigma_2 = 0.7811$, $\sigma_3 = 3.9 \times 10^{-7} \rightarrow$ effective rank = 2

- The effective rank is the no. of singular values that are greater than the noise level.

For Personal Use Only -bkwk2

Least squares solution — full rank case.

- If A is a $m \times n$ matrix w/ $m > n$, we can partition the SVD as

$$A = [U_1 \ U_2] \begin{bmatrix} \Sigma_1 \\ 0 \end{bmatrix} V^H = U_1 \Sigma_1 V_1^H$$

since A is full rank, Σ_1 will be $n \times n$, and U_1 is $m \times n$.

* Note the columns of U are orthonormal $\rightarrow U_1^H U_1 = I_{nn}$ and $U_2^H U_1 = 0_{(m-n) \times n}$

- A least squares soln minimises $r = \|Ax - b\|_2^2$

$$r = \|Ax - b\|_2^2 = \left\| \begin{bmatrix} U_1^H \\ U_2^H \end{bmatrix} (Ax - b) \right\|_2^2 = \left\| \begin{bmatrix} U_1^H \\ U_2^H \end{bmatrix} (U_1 \Sigma_1 V_1^H x - b) \right\|_2^2 = \left\| \begin{bmatrix} \Sigma_1 V_1^H x - U_1^H b \\ -U_2^H b \end{bmatrix} \right\|_2^2 = \underbrace{\|\Sigma_1 V_1^H x - U_1^H b\|_2^2}_{\text{independent of } x} + \|U_2^H b\|_2^2$$

The residual r is minimised when $\Sigma_1 V_1^H x = U_1^H b$, i.e.

$$x = V_1 \Sigma_1^{-1} U_1^H b$$

Both Σ_1 and U_1 are full rank, so the least squares soln is unique. We could add zero-padding to U_1, Σ_1 ,

$$\hat{x} = V \Sigma^+ U^H b, \quad \text{where } \Sigma^+ = \begin{bmatrix} 1/\sigma_1 & \dots & 1/\sigma_n \\ 0 & \dots & 0 \end{bmatrix} \text{ is the pseudoinverse of } \Sigma$$

- We can rederive the least squares expression by expanding $\hat{x} = A^+ b$, $A^+ = (A^H A)^+ A^H$, $A = U_1 \Sigma_1 V_1^H$

$$A^+ = (A^H A)^+ A^H = (V_1 \Sigma_1^H U_1^H U_1 \Sigma_1 V_1^H)^+ V_1 \Sigma_1^H U_1^H = (V_1 \Sigma_1^H \Sigma_1 V_1^H)^+ V_1 \Sigma_1^H U_1^H$$

$$= V_1 \Sigma_1^{-1} \Sigma_1^H V_1 \Sigma_1^H U_1^H = V_1 \Sigma_1^{-1} \Sigma_1^H \Sigma_1^H U_1^H = V_1 \Sigma_1^{-1} U_1^H \rightarrow \hat{x} = V_1 \Sigma_1^{-1} U_1^H b = V \Sigma^+ U^H b$$

- Consider the ℓ_2 norm of the least squares soln, squared, $\|\hat{x}\|_2^2$

$$\|\hat{x}\|_2^2 = (V \Sigma^+ U^H b)^H V \Sigma^+ U^H b = b^H U \Sigma^+ U^H b = \| \Sigma^+ U^H b \|_2^2 \geq |(\Sigma^+ U^H b)_n|^2 = \frac{|U_n^H b|^2}{\sigma_n^2}$$

i.e. if the smallest singular value $\sigma_{\min}$ is small, then the least squares soln will be large and sensitive to changes in $b \rightarrow$ poor conditioning // If $\sigma_{\min} = 0$, we cannot compute a unique least squares soln.

Least squares solution — rank deficient case.

- If A is a matrix w/ zero singular values, we can partition the SVD as

$$A = [U_1 \ U_2] \begin{bmatrix} \Sigma_1 & 0 \\ 0 & 0 \end{bmatrix} [V_1 \ V_2]^H = U_1 \Sigma_1 V_1^H$$

where Σ_1 has size $r \times r$, r is the rank of the matrix A .

* Note the columns of U are orthonormal $\rightarrow U_1^H U_1 = I_{rr}$ and $U_2^H U_1 = 0_{(m-r) \times r}$

- A least squares soln minimises $r = \|Ax - b\|_2^2$

$$r = \|Ax - b\|_2^2 = \left\| \begin{bmatrix} U_1^H \\ U_2^H \end{bmatrix} (Ax - b) \right\|_2^2 = \left\| \begin{bmatrix} U_1^H \\ U_2^H \end{bmatrix} (U_1 \Sigma_1 V_1^H x - b) \right\|_2^2 = \left\| \begin{bmatrix} \Sigma_1 V_1^H x - U_1^H b \\ -U_2^H b \end{bmatrix} \right\|_2^2 = \underbrace{\|\Sigma_1 V_1^H x - U_1^H b\|_2^2}_{\text{independent of } x} + \|U_2^H b\|_2^2$$

The residual r is minimised when $\Sigma_1 V_1^H x = U_1^H b$, i.e.

$$\hat{x} = V_1 \Sigma_1^{-1} U_1^H b$$

- Note V_1 is only a part of $V \rightarrow$ has a non-trivial null-space, specifically $V_1^H (V_2 z) = 0$ for all $z \in \mathbb{C}^{m-r}$

since the columns of V_1 and V_2 are orthogonal, we thus have infinitely many soln satisfying $\Sigma_1 V_1^H x = U_1^H b$

$$\hat{x} = V_1 \Sigma_1^{-1} U_1^H b + V_2 z$$

Taking the norm and noting that V_1 and V_2 are orthogonal wrt each other,

$$\|\hat{x}\|_2^2 = \|V_1 \Sigma_1^{-1} U_1^H b + V_2 z\|_2^2 = \|V_1 \Sigma_1^{-1} U_1^H b\|_2^2 + \|V_2 z\|_2^2$$

$\therefore \hat{x} = V_1 \Sigma_1^{-1} U_1^H b$ is the minimiser to the least squares problem w/ minimal ℓ_2 norm

For Personal Use Only -bkwk2

Fitting points and singular systems.

- To solve for the coefficients of the interpolating polynomial, we need to solve the matrix eqn. $A\mathbf{c} = \mathbf{f}$.

If the data points happen to lie on a line, then A will be singular.

- When A is singular, $A^H A$ is also singular, i.e. $(A^H A)^{-1}$ doesn't exist $\rightarrow$ cannot find the least squares soln by solving $A^H A \mathbf{c} = A^H \mathbf{f}$.

- To find the least squares soln, we perform SVD of A and compute $\mathbf{c} = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^H \mathbf{f}$

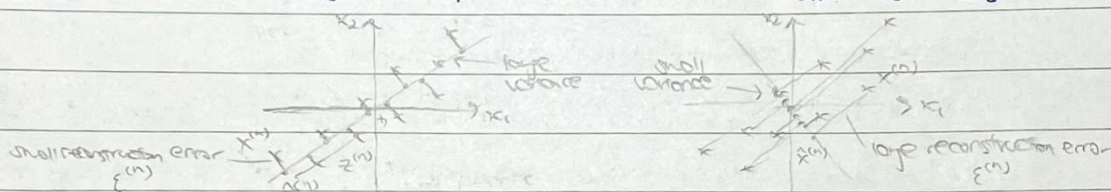
- If the matrix A is near singular, we compute the SVD, ignore singular values less than the noise level, then compute $\mathbf{c} = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^H \mathbf{f}$ as usual. (unstable if we do not ignore those singular values).

Principal component analysis (PCA)

Principal component analysis (PCA).

- PCA is a dimension reduction technique. It finds new, uncorrelated variables from a dataset.

- The "new" variables are the dir. along which the variance of the sample is greatest / the reconstruction error is smallest. They can be defined as linear combination of the original variables.



- consider projecting from $\mathbf{x}^{(n)} \in \mathbb{R}^D$ to $\mathbf{z}^{(n)} \in \mathbb{R}^M$, w/ $M < D$. choose M projection vectors $\mathbf{w}_1, \dots, \mathbf{w}_M \in \mathbb{R}^D$, s.t. they are unit length $\|\mathbf{w}_i\| = 1$ and orthogonal, $\mathbf{w}_i^T \mathbf{w}_j = 0 \ \forall i \neq j$. Also $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}$ have zero mean.

The projection of the n th vec $\mathbf{x}^{(n)}$ onto the M th dimension is $\mathbf{z}_M^{(n)} = \mathbf{w}_M^T \mathbf{x}^{(n)}$, so in general, $\mathbf{x}^{(n)}$ is mapped to

$$\mathbf{z}^{(n)} = \begin{bmatrix} \mathbf{z}_1^{(n)} \\ \vdots \\ \mathbf{z}_M^{(n)} \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^T \mathbf{x}^{(n)} \\ \vdots \\ \mathbf{w}_M^T \mathbf{x}^{(n)} \end{bmatrix} = \begin{bmatrix} \leftarrow \mathbf{w}_1^T \rightarrow \\ \vdots \\ \leftarrow \mathbf{w}_M^T \rightarrow \end{bmatrix} \begin{bmatrix} x_1^{(n)} \\ \vdots \\ x_D^{(n)} \end{bmatrix} = \mathbf{W}^T \mathbf{x}^{(n)}$$

For reconstruction, we have $\hat{\mathbf{x}}^{(n)} = \mathbf{W} \mathbf{z}^{(n)}$

- The (j,k) th entry of the sample covariance matrix is given by $s_{jk} = \frac{1}{N-1} \sum_{n=1}^N x_{j1} x_{k1}$, given $\bar{x}_i = 0 \ \forall i$

In matrix form, the sample covariance matrix is given by $\mathbf{S} = \frac{1}{N-1} \mathbf{X}^T \mathbf{X}$, for $\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \vdots \\ \mathbf{x}^{(n)T} \end{bmatrix}$

- The sample variance of direction $\mathbf{w}_1$, $\hat{\sigma}_{z_1}^2$, is given by $\hat{\sigma}_{z_1}^2 = \frac{1}{N-1} \sum_{n=1}^N (\mathbf{z}_1^{(n)} - \bar{\mathbf{z}}_1)^2$

$$\hat{\sigma}_{z_1}^2 = \frac{1}{N-1} \sum_{n=1}^N (\mathbf{z}_1^{(n)} - \bar{\mathbf{z}}_1)^2 = \frac{1}{N-1} \sum_{n=1}^N (\mathbf{x}^{(n)T} \mathbf{w}_1)^2 = \frac{1}{N-1} \sum_{n=1}^N \mathbf{w}_1^T \mathbf{x}^{(n)} \mathbf{x}^{(n)T} \mathbf{w}_1 = \mathbf{w}_1^T \left(\frac{1}{N-1} \sum_{n=1}^N \mathbf{x}^{(n)} \mathbf{x}^{(n)T} \right) \mathbf{w}_1 = \mathbf{w}_1^T \mathbf{S} \mathbf{w}_1$$

To max. this quantity, we choose $\mathbf{w}_1$ to be the eigenvector of $\mathbf{S} = \frac{1}{N-1} \mathbf{X}^T \mathbf{X}$ that corresponds to the largest eigenvalue.

(we find $\mathbf{w}_2, \mathbf{w}_3$ by finding the eigenvector of $\mathbf{S}$ that corresponds to the second/third largest eigenvalue)

- Note that the eigenvalues and eigenvectors of $\mathbf{X}^T \mathbf{X}$ can be computed using the SVD of $\mathbf{X}$:

$\hookrightarrow$ eigenvalues of $\mathbf{X}^T \mathbf{X}$ are the squared singular values of $\mathbf{X}$ (diagonals of $\mathbf{\Sigma}^2$).

$\hookrightarrow$ eigenvectors of $\mathbf{X}^T \mathbf{X}$ are the columns of the unitary matrix $\mathbf{V}$

Finite-space Markov chains

Markov chains

- A stochastic process $X_0, X_1, \dots$ is a Markov chain iff for all times $i \geq 1$

$$P(X_{i+1} | X_0 = j_0, X_1 = j_1, \dots, X_i = j_i) = P(X_{i+1} | X_i = j_i)$$

where each time-indexed RV $X_0, X_1, \dots$ takes a value from the state space S ($j \in S \forall i$)

- A Markov chain can be described by the transition matrix $\underline{P}$, where the (j,k) th element satisfies

$$P_{j,k} = P[\text{state } j \rightarrow \text{state } k]$$

$$\sum_k P_{j,k} = 1$$

- Denoting the distribution of states at time n by $\underline{x}^{(n)}$ and the initial distribution by $\underline{x}^{(0)}$, we have

$$\underline{x}^{(n+1)} = \underline{x}^{(n)} \underline{P}$$

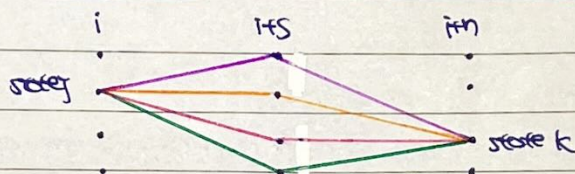
$\rightarrow$

$$\underline{x}^{(n)} = \underline{P}^n \underline{x}^{(0)}$$

i.e. we can get the n -step transition matrix by raising the 1-step transition matrix to the power n

- Using the notation $(\underline{P}^n)_{j,k} = P_{j,k}^{(n)}$, we can derive the Chapman-Kolmogorov eqn as follows:

$$P[X_{i+n} = k | X_i = j] = (\underline{P}^n)_{j,k} = (\underline{P}^S \underline{P}^{n-S})_{j,k} = \sum_l P_{j,l}^S P_{l,k}^{n-S} = \sum_l P[X_{i+S} = l | X_i = j] P[X_{i+n} = k | X_{i+S} = l]$$



Stationary distributions and limiting distributions

- For a Markov process w/ transition matrix $\underline{P}$, a stationary distribution is any distribution $\underline{\pi}$.

$$\underline{\pi}^{(0)} \underline{P} = \underline{\pi}^{(n)}$$

- A limiting distribution is one which for any initial distribution $\underline{x}^{(0)}$ satisfies

$$\underline{x}^{(0)} \underline{P}^n = \underline{\pi}^{(n)} \quad \text{as } n \rightarrow \infty$$

(A limiting distribution is the unique stationary position of a Markov process).

Transition matrix eigenvalues

- For a $n \times n$ transition matrix $\underline{P}$, (i) $\lambda = 1$ is an eigenvalue, (ii) all eigenvalues satisfy $|\lambda| \leq 1$.

$\hookrightarrow$ (i) $\lambda = 1$ is an eigenvalue.

$$\underline{P} - \underline{I} = \begin{bmatrix} P_{1,1}-1 & P_{1,2} & \dots & P_{1,N} \\ P_{2,1} & P_{2,2}-1 & \dots & P_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ P_{N,1} & P_{N,2} & \dots & P_{N,N}-1 \end{bmatrix}$$

Sum of each row is $1-1=0$

$\rightarrow$ summing first $N-1$ col yields we lost col

$\therefore \underline{P} - \underline{I}$ is not full rank $\rightarrow \det(\underline{P} - \underline{I}) = 0 \rightarrow \lambda = 1$ is an eigenvalue.

$\hookrightarrow$ (ii) All eigenvalues satisfy $|\lambda| \leq 1$

select largest magnitude element of eigenvector $\underline{v}_k$, i.e. $|v_k| \geq |v_i| \forall i$

$$|\lambda| |v_k| = |\lambda \underline{v}_k| = |(\underline{P} \underline{v}_k)_k| = |P_{k,1} v_1 + \dots + P_{k,N} v_N| \leq P_{k,1} |v_1| + \dots + P_{k,N} |v_N|$$

$$\leq P_{k,1} |v_k| + \dots + P_{k,N} |v_k| = (P_{k,1} + \dots + P_{k,N}) |v_k| = |v_k| \rightarrow |\lambda| \leq 1$$

+ we can find the stationary distribution by finding the left eigenvector corr. to $\lambda = 1 \quad \underline{v} \underline{P} = \underline{v}$.

For Personal Use Only -bkwk2

Irreducible, recurrence and transience.

- Two states of a Markov chain j and k communicate iff there exist integers m, n s.t.

$$P_{j,k}^{(m)} > 0 \quad \text{and} \quad P_{k,j}^{(n)} > 0$$

i.e. there must be a route from state j to k in m steps

best determine by
drawing up state diagram
↓

- If all states in a Markov chain communicate w/ each other, then the Markov chain is irreducible.

- Let S denote the set of all possible states of a Markov chain. A subset $\tilde{S} \subseteq S$ is a recurrent set if (i) all pairs of states in $\tilde{S}$ communicate (ii) if $j \in \tilde{S}, k \notin \tilde{S}$, then $P_{j,k}^i = 0$ for all i
i.e. it is possible to move between all states in a recurrent set, but cannot move from a member of a recurrent set to not a member.

- If a state belongs to a recurrent set, then it is recurrent, o/w it is transient.

↳ If state k is recurrent, then $P[\text{return to state } k | X_0 = k] = 1$

↳ If we start in a transient state, then we will leave that state and never return to it.

Periodicity

- The period of a state k of a Markov chain is the greatest common divisor of the set

$$\{r \geq 0, P_{k,k}^{(r)} > 0\}$$

(eg. $\{18, 24\} \rightarrow 18 = 2 \times 3^2, 24 = 2^3 \times 3$ so $\text{GCD}(18, 24) = 2 \times 3 = 6$)

- A state k is aperiodic if it has period 1. This is true if for some time i

$$P_{k,k}^{(i)} > 0$$

and

$$P_{k,k}^{(i+1)} > 0$$

- A Markov chain is aperiodic if all states are aperiodic.

- If the Markov chain is irreducible, all the states have the same period.

(All states in a recurrent set have the same period)

- For an irreducible Markov chain w/ period d , there will be d eigenvalues of P w/ magnitude 1 (d -roots of unity)

Regular ergodicity

Every regular MC is ergodic
but an ergodic MC may not be regular
eg. $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

- A Markov chain is regular if for some integer n , all entries of P^n are positive.

An ergodic Markov chain is equivalent to an irreducible Markov chain,

- If a Markov chain is irreducible and aperiodic, then it is regular ergodic.

↳ A regular ergodic Markov chain has a limiting distribution which puts the mass on every element of the state space S

↳ If $X_0, X_1, \dots$ is a regular ergodic Markov chain w/ limiting distribution π , then for any function $f(x)$, as $N \rightarrow \infty$, $\frac{1}{N} \sum_{i=1}^N f(X_i) \rightarrow \sum_{k \in S} \pi(k) f(k)$

For Personal Use Only -bkwk2

Detailed balance.

- A transition matrix $\underline{P}$ and a distribution $\underline{\pi}$ are in detailed balance if

$$\pi_j P_{j,k} = \pi_k P_{k,j} \quad \forall j,k$$

Consider this distribution after one step, $\tilde{\pi} = \underline{\pi} \underline{P}$.

$$\tilde{\pi}_k = \sum_j \pi_j P_{j,k} = \sum_j \pi_k P_{k,j} = \pi_k \left(\sum_j P_{k,j} \right) = \pi_k \rightarrow \tilde{\pi} = \underline{\pi}$$

$\therefore \underline{\pi}$ is a stationary distribution of $\underline{P}$.

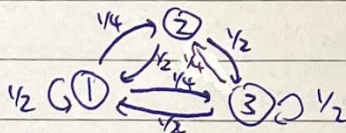
* If $\underline{\pi}$ satisfies detailed balance w.r. $\underline{P}$, $\underline{\pi}$ is a stationary distribution for the transition matrix $\underline{P}$, but a stationary distribution of $\underline{P}$ does not necessarily satisfy detailed balance.

Waiting time

- To calculate the expected waiting time to reach a certain state k from state j , q_j , we set up an expression q_j for each of the N states and solve.

- e.g. Consider a Markov chain w.r. transition matrix $\underline{P}$, ($N=3$)

$$\underline{P} = \begin{bmatrix} 1/2 & 1/4 & 1/4 \\ 1/2 & 0 & 1/2 \\ 1/4 & 1/4 & 1/2 \end{bmatrix}$$



Find the average wait time to transition to state 2, given we start at state 2.

$$\begin{cases} q_1 = \frac{1}{2}(1+q_1) + \frac{1}{4}(1) + \frac{1}{4}(1+q_1) \\ q_2 = \frac{1}{2}(1+q_1) + \frac{1}{2}(1+q_3) \\ q_3 = \frac{1}{4}(1+q_1) + \frac{1}{4}(1) + \frac{1}{2}(1+q_2) \end{cases} \rightarrow q_1=4, q_2=5, q_3=4$$

$\therefore$ average wait time req. is 5 steps.

Continuous state space systems

Birth process

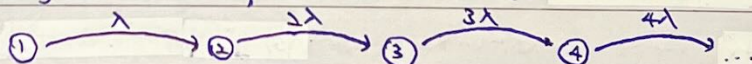
- Let $n(t)$ be the no. of cells at time t , $n(0)=n_0$ and λ be the birth rate for a cell per unit time.

$n(t)$ describes the state of the system $\rightarrow$ Markovian. For a small time step Δt ,

$$n(t+\Delta t) = n(t) + n(t)\lambda \Delta t \xrightarrow{\Delta t \rightarrow 0} \frac{dn(t)}{dt} = \lambda n(t) \rightarrow n(t) = n_0 \exp(\lambda t)$$

* $n(t)$ gives the expected value of the no. of cells $\rightarrow$ consider probabilistic approach for distribution of n .

- Let $P_n(t) = P[n(t)=n]$ be the probability of n cells at time t .



$$\text{For a small time step } \Delta t, \quad P_n(t+\Delta t) = P_n(t)(1-n\lambda\Delta t) + P_{n-1}(t)(n-1)\lambda\Delta t$$

As $\Delta t \rightarrow 0$,

$$\frac{dP_n(t)}{dt} = -n\lambda P_n(t) + (n-1)\lambda P_{n-1}(t)$$

- We can represent the system in terms of the transition rate matrix $\underline{Q}$

$$\frac{d\underline{x}(t)}{dt} = \underline{x}(t) \underline{Q}$$

rows sum to zero

where $\underline{x}(t) = [P_1(t) \dots P_n(t) \dots]$, $\sum_{n=1}^{\infty} P_n(t) = 1$ and $\underline{Q} = \begin{bmatrix} -\lambda & \lambda & 0 & 0 & \dots \\ 0 & -2\lambda & 2\lambda & 0 & \dots \\ 0 & 0 & -3\lambda & 3\lambda & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$

For Personal Use Only -bkwk2

- It can be shown that the solution to the birth process is

$$P_n(t) = C_{n-1}^{n-1} \exp(-\lambda n_0 t) (1 - \exp(-\lambda t))^{n-n_0}$$

- If the birth process did reach a steady state, then

$$\frac{d\mathbf{x}(\infty)}{dt} = \mathbf{x}(\infty) \underline{Q} = \underline{0}$$

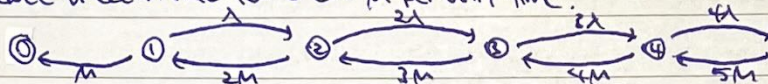
$$\rightarrow \begin{cases} 0 = -\lambda x_1(\infty) \\ 0 = \lambda x_1(\infty) - 2\lambda x_2(\infty) \\ \vdots \\ 0 = (n-1)x_{n-1}(\infty) - n\lambda x_n(\infty) \end{cases}$$

the set of eqns are only satisfied when $x_i(\infty) = 0$ for all $i \rightarrow$ not a valid PDF

$\therefore$ There is no steady state distribution (the no. of cells keep growing)

Birth-death process

- Now we introduce a death rate for a cell μ per unit time.



For a small time step Δt , $P_n(t+\Delta t) = P_n(t)(1 - n\lambda\Delta t - n\mu\Delta t) + P_{n-1}(t)(n-1)\lambda\Delta t + P_{n+1}(t)(n+1)\mu\Delta t$

As $\Delta t \rightarrow 0$,

$$\frac{dP_n(t)}{dt} = -n(\lambda + \mu)P_n(t) + (n-1)\lambda P_{n-1}(t) + (n+1)\mu P_{n+1}(t)$$

- We can represent the system in terms of the transition rate matrix $\underline{Q}$.

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{x}(t) \underline{Q}$$

where $\mathbf{x}(t) = [P_0(t) \dots P_n(t) \dots]$, $\sum_{n=0}^{\infty} P_n(t) = 1$ and $\underline{Q} = \begin{bmatrix} 0 & \lambda & 0 & 0 & \dots \\ \mu & -(\lambda + \mu) & \lambda & 0 & \dots \\ 0 & \mu & -2(\lambda + \mu) & 2\lambda & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$

- For steady state, we req.

$$\frac{d\mathbf{x}(\infty)}{dt} = \mathbf{x}(\infty) \underline{Q} = \underline{0} \rightarrow -n(\lambda + \mu)P_n + (n-1)\lambda P_{n-1} + (n+1)\mu P_{n+1} = 0$$

It can be shown that $\mathbf{x}(\infty) = [1 \ 0 \ 0 \ \dots]$ $\rightarrow$ all states except 0 are transient.

* If we λ_i, μ_i be the birth/death rate per unit time for state i respectively, then it can be shown using recursion that for $i > 0$, $P_i(\infty) = \left(\frac{\prod_{j=0}^{i-1} \lambda_j}{\prod_{j=1}^i \mu_j} \right) P_0(\infty)$

Random walk

- Consider a random walk - at each time, we take step ± 1 , where the probability of dir.

at time k $S_k \in \{-1, 1\}$ is uniform, i.e. $P(S_k = 1) = P(S_k = -1) = 1/2$.

- After n steps, the position X_n is given by

$$X_n = \sum_{k=1}^n S_k$$

- At each instance, the average step is 0, and step variance is 1 $\rightarrow$ by CLT, the distribution of X_n as $N \rightarrow \infty$ is a Gaussian $N(0, N)$.

- If we now take a small step δ in dir. S_k every δ seconds, the distribution of the position W_t

at time $t = N\delta$ as $N \rightarrow \infty$ is a Gaussian $N(0, t) = N(0, N\delta)$

(As $\delta \rightarrow 0$, we have Brownian motion)

For Personal Use Only -bkwk2

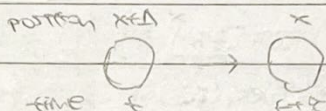
Brownian motion

- Brownian motion is the random motion of particles in a fluid resulting from collisions. since there are too many particles, we consider the particle density.
- Consider a simplified system where each dir. is independent, where the overall form is obtained by simply multiplying dimensions together. $\rightarrow$ only consider x dimension
- Let the particle density at time t and position x be $f(x, t)$. We can define the state of the system since probability of the next state only depends on the current state of time $t \rightarrow$ Markov process.
- Assuming smoothness in the particle density $f(x)$ over position x and time t , we can write

$$f(x+\Delta, t) = f(x, t) + \Delta \frac{\partial f(x, t)}{\partial x} + \frac{\Delta^2}{2} \frac{\partial^2 f(x, t)}{\partial x^2} + O(\Delta^3) \quad [1]$$

$$f(x, t+\tau) = f(x, t) + \tau \frac{\partial f(x, t)}{\partial t} + O(\tau^2) \quad [2]$$

- The change in density at time $t+\tau$ results from particles moving to position x from time t to $t+\tau$.



Considering all initial positions $x+\Delta$, the density at time $t+\tau$ is given by

$$f(x, t+\tau) = \int_{-\infty}^{\infty} f(x+\Delta, t) P(\Delta) d\Delta \quad [3]$$

where $P(\Delta)$ is the probability of a particle moving Δ in time τ . continuous form of Chapman-Kolmogorov

- Sub [1] into [3], and by symmetry $P(\Delta) = P(-\Delta)$,

$$\begin{aligned} f(x, t+\tau) &\approx \int_{-\infty}^{\infty} \left[f(x, t) + \Delta \frac{\partial f(x, t)}{\partial x} + \frac{\Delta^2}{2} \frac{\partial^2 f(x, t)}{\partial x^2} \right] P(\Delta) d\Delta \\ &= f(x, t) \int_{-\infty}^{\infty} P(\Delta) d\Delta + \frac{\partial f(x, t)}{\partial x} \int_{-\infty}^{\infty} \Delta P(\Delta) d\Delta + \frac{1}{2} \frac{\partial^2 f(x, t)}{\partial x^2} \int_{-\infty}^{\infty} \Delta^2 P(\Delta) d\Delta \\ &= f(x, t) + \frac{1}{2} \frac{\partial^2 f(x, t)}{\partial x^2} \int_{-\infty}^{\infty} \Delta^2 P(\Delta) d\Delta \end{aligned}$$

Comparing this w/ [2], we have

$$f(x, t+\tau) = f(x, t) + \tau \frac{\partial f(x, t)}{\partial t} = f(x, t) + \frac{1}{2} \frac{\partial^2 f(x, t)}{\partial x^2} \int_{-\infty}^{\infty} \Delta^2 P(\Delta) d\Delta$$

$$\therefore \tau \frac{\partial f(x, t)}{\partial t} = \frac{1}{2} \frac{\partial^2 f(x, t)}{\partial x^2} \int_{-\infty}^{\infty} \Delta^2 P(\Delta) d\Delta \rightarrow \frac{\partial f(x, t)}{\partial t} = D \frac{\partial^2 f(x, t)}{\partial x^2} \quad \text{where } D = \frac{1}{2\tau} \int_{-\infty}^{\infty} \Delta^2 P(\Delta) d\Delta$$

- The value of D can be determined by measurement of physical properties $\rightarrow$ no need to consider τ or $P(\Delta)$.
- The final sol'n depends on the initial conditions $\rightarrow$ particular sol'n.

solution for Brownian motion differential equation.

- Consider the Brownian motion DE for a const. α ,

$$\frac{\partial p(x,t)}{\partial t} = \alpha \frac{\partial^2 p(x,t)}{\partial x^2}$$

Assuming the sol'n can be written in the form, $p(x,t) = X(x)T(t)$,

$$X \frac{dT}{dt} = \alpha T \frac{d^2 X}{dx^2} \rightarrow \frac{1}{T} \frac{dT}{dt} = \frac{1}{X} \frac{d^2 X}{dx^2} = -k^2 \leftarrow \text{separation const.}$$

$$\therefore X(x) = A \sin(kx) + B \cos(kx), \quad T(t) = C \exp(-\alpha k^2 t) \rightarrow p(x,t) = A(k) \exp(-\alpha k^2 t) \exp(i k x)$$

The above sol'n is satisfied by any k , so the general sol'n is

$$p(x,t) = \int_{-\infty}^{\infty} A(k) \exp(-\alpha k^2 t) \exp(i k x) dk$$

- consider the initial condition as $p(x,0) = \delta(x)$

$$\delta(x) = p(x,0) = \int_{-\infty}^{\infty} A(k) \exp(i k x) dk = \int_{-\infty}^{\infty} A(\tilde{k}) \exp(i \tilde{k} x) d\tilde{k}$$

Noting that this is the FT of $A(\tilde{k})$, i.e. $FT[A(\tilde{k})] = \delta(x)$, so

$$A(\tilde{k}) = \text{IFT}[\delta(x)] = \frac{1}{2\pi}$$

$$\therefore p(x,t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp(-\alpha k^2 t) \exp(i k x) dk = \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp(-\alpha k^2 t + i k x) dk$$

complete the square in k .

The term in the exponent can be rewritten as $-\alpha k^2 t + i k x = -(\sqrt{\alpha t} k - \frac{i k x}{2\sqrt{\alpha t}})^2 - \frac{x^2}{4\alpha t}$

$$p(x,t) = \frac{1}{2\pi} \exp(-\frac{x^2}{4\alpha t}) \int_{-\infty}^{\infty} \exp[-(\sqrt{\alpha t} k - \frac{i k x}{2\sqrt{\alpha t}})^2] dk = \frac{1}{2\pi} \exp(-\frac{x^2}{4\alpha t}) \int_{-\infty}^{\infty} \exp(-u^2) du$$

$$p(x,t) = \frac{1}{\sqrt{4\pi\alpha t}} \exp(-\frac{x^2}{4\alpha t}) = N(x|0, 2\alpha t)$$

properties of a 1D Wiener process.

- The simplified Brownian motion is a 1D Wiener process. At time t , examine the Wiener process W_t .

We can consider an instance of a "path" at time t $W_t^{(i)}$ (the support of the RV X_t).

- The Wiener process $\{W_t\}_{t \geq 0}$ has the following properties:

(i) Stationarity: $W_{t+s} - W_s$ is independent of s , e.g. $W_t = W_{t+s} - W_s$

(ii) Independence: $W_t - W_s$ is independent of $\{W_u\}_{u \leq s}$, e.g. $W_0 - W_s \perp W_s$

(iii) Continuity: W_t is a continuous function w/ t

(iv) Gaussianity: W_t is a Gaussian w/ $E[W_t] = 0$, $E[W_t W_s] = 2\alpha \min(t, s)$

general continuous state-space systems

- A range of assumptions have been made in the previous derivation - some may not be valid.

- A simple extension is to include a drift term: Wiener process w/ drift.

$$\frac{\partial p(x,t)}{\partial t} = \mu \frac{\partial p(x,t)}{\partial x} + D \frac{\partial^2 p(x,t)}{\partial x^2}$$

- More complex models include the Ornstein-Uhlenbeck process.

$$\frac{\partial p(x,t)}{\partial t} = \frac{\partial (\beta x p(x,t))}{\partial x} + \frac{\partial^2 (\sigma^2 p(x,t))}{\partial x^2}$$

where βx controls the drift and σ^2 controls the diffusion

* When $\beta = 0$, we have the Wiener process. When $\beta \neq 0$, the diffusion term can be balanced by the drift term so the particle density tends to a limiting distribution (finite variance)

For Personal Use Only -bkwk2

Monte Carlo methods

Monte Carlo integration.

- consider a highly complicated, multi-dimensional ($x \in \mathbb{R}^d$) integral $\int h(x) dx$.

using a histogram approach (in 1 dimension), we have

$$\int h(x) dx \approx \sum_{i=1}^N h(x^{(i)}) \delta x_1 \delta x_2 \dots \delta x_d$$

where we have uniformly spread N histogram centres.

- The histogram approach is simple but wasteful for regions w/ very small (no) contributions to the integral. It also does not scale well w/ the no. of dimensions.
 to not lose accuracy in the approx, we req. a bins per dimension $\rightarrow n^d$ bins for d dimensions.

- For Monte Carlo integration, we sample from the normalised weighting function $w(x)$, $p(x)$

$$p(x) = \frac{w(x)}{\int w(x) dx}$$

This is used to model the distribution of the pts.

- Initially consider uniform distribution over a "volume" V , $p(x) = \frac{1}{V}$. The integral becomes.

$$\int h(x) dx = \int \frac{h(x)}{p(x)} p(x) dx = E_p \left[\frac{h(x)}{p(x)} \right] \approx \frac{1}{N} \sum_{i=1}^N \frac{h(x^{(i)})}{p(x^{(i)})} = \frac{V}{N} \sum_{i=1}^N h(x^{(i)})$$

where $x^{(i)}$ are drawn from $p(x)$.

- selecting an appropriate "volume" for $p(x)$ avoids wasted samples $\rightarrow$ smaller variance for a fixed no. of samples.

Monte Carlo for expected values.

- consider computing the expected value of a function $f(x)$, $E[f(x)]$. The integral has the form

$$\int f(x) p(x) dx = E[f(x)] \approx \frac{1}{N} \sum_{i=1}^N f(x^{(i)})$$

where $x^{(i)}$ are drawn from $p(x)$.

- denote the mean of the estimate using 1 sample w/ $\hat{M}_1$, $\hat{M}_1 = \int f(x) p(x) dx$.

The mean of the estimate using N samples, $\hat{M}_N$ is given by

$$\hat{M}_N = \frac{1}{N} \sum_{i=1}^N \hat{M}_1 = \hat{M}_1 \leftarrow \text{unbiased.}$$

- denote the variance of the estimate using 1 sample w/ $\hat{\sigma}_1^2$, $\hat{\sigma}_1^2 = \int (f(x) - M)^2 p(x) dx$.

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{i=1}^N \hat{\sigma}_1^2 = \frac{1}{N} \hat{\sigma}_1^2$$

- In fact, for large N , the distribution of $\frac{1}{N} \sum_{i=1}^N f(x^{(i)}) \rightarrow N(\hat{M}_N, \frac{\hat{\sigma}_1^2}{N}) = N(\hat{M}_1, \frac{\hat{\sigma}_1^2}{N})$ by CLT.

$\rightarrow$ the approach yields correct answers (as $N \rightarrow \infty$).

For Personal Use Only -bkwk2

Importance sampling.

- If we want to sample from another pdf $q(x)$ rather than $p(x)$ (say it is not possible to sample from $p(x)$ or we want to sample from a non-uniform distribution), we use importance sampling.
- Importance sampling can be used for computing expected values / numerical integration:

① computing expected value $E_p[f(x)]$

$$E_p[f(x)] = \int f(x)p(x)dx = \int f(x)\frac{p(x)}{q(x)}q(x)dx = E_q\left[f(x)\frac{p(x)}{q(x)}\right] \approx \frac{1}{N}\sum_{i=1}^N f(x^{(i)})\frac{p(x^{(i)})}{q(x^{(i)})}$$

↳ the ratio $\frac{p(x^{(i)})}{q(x^{(i)})}$ is the importance of the drawn sample ← how much to "boost" a sample $f(x^{(i)})$

($q(x) > p(x)$: over-represented region ; $p(x) > q(x)$: under-represented region)

↳ we can perform importance sampling w/o the normalised distributions. for $x^{(i)} \sim q(x)$.

let $p(x) = \frac{1}{Z_p} \tilde{p}(x)$, $q(x) = \frac{1}{Z_q} \tilde{q}(x)$, $w^{(i)} = \frac{\tilde{p}(x^{(i)})}{\tilde{q}(x^{(i)})}$, where Z_p, Z_q cannot be computed.

The expected value can be shown to be $V = \int f(x)\frac{p(x)}{q(x)}q(x)dx \approx \frac{\sum_{i=1}^N f(x^{(i)})w^{(i)}}{\sum_{i=1}^N w^{(i)}}$

(Numerator: $\frac{1}{N}\sum_{i=1}^N f(x^{(i)})w^{(i)} \approx E_q[f(x)w^{(i)}] = \int f(x)\frac{\tilde{p}(x)}{\tilde{q}(x)}q(x)dx = \frac{Z_p}{Z_q} \int f(x)p(x)dx = \frac{Z_p}{Z_q} V$

Denominator: $\frac{1}{N}\sum_{i=1}^N w^{(i)} \approx E_q[w^{(i)}] = \int \frac{\tilde{p}(x)}{\tilde{q}(x)}q(x)dx = \frac{Z_p}{Z_q} \int p(x)dx = \frac{Z_p}{Z_q}$)

② Numerical integration $\int h(x)dx$

$$\int h(x)dx = \int \frac{h(x)}{q(x)}q(x)dx = E_q\left[\frac{h(x)}{q(x)}\right] \approx \frac{1}{N}\sum_{i=1}^N \frac{h(x^{(i)})}{q(x^{(i)})}$$

↳ The ratio $\frac{h(x^{(i)})}{q(x^{(i)})}$ is similar to the "importance" of the drawn sample.

↳ we want $q(x)$ to be close to $h(x)$ → importance ≈ 1 everywhere

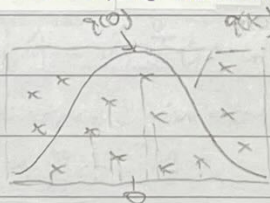
if $q(x) \propto h(x)$, then $\frac{h(x)}{q(x)}$ is a const. → we can get the answer in 1 sample.

↑ Using importance sampling may give a smaller variance in the estimate for a fixed no. of samples.

Generating random samples - rejection sampling.

- We want to generate random samples from the distribution $q(x)$, where
- In rejection sampling, we enclose (more or) the distribution $q(x)$ in a 2D box

assume we can generate uniform RVs



Drawing samples uniformly from the 2D box, we accept those under the curve (take x-values) and reject those above the line

- The fraction of samples accepted is given by

$$\text{fraction of samples accepted} = \frac{\text{area under curve}}{\text{area of box}}$$

← slightly under 1.

For higher dimensions, $x \in \mathbb{R}^d$, the fraction of samples accepted would be raised to the power of d → highly wasteful (curse of dimensionality).

For Personal Use Only -bkwk2

Generating random samples — Metropolis-Hastings algorithm.

- We want to generate random samples from the distribution $p(x)$, i.e. $\mathcal{P}$.

- The Metropolis-Hastings algorithm is as follows:

→ given the current sample $x^{(i)}$, generate another sample x^* from $p(x|x^{(i)})$

$$x^* = x^{(i)} + z$$

where z is the proposal distribution (e.g. $z \sim N(0, I)$)

→ accept the sample, i.e. $x^{(i+1)} = x^*$ w/ probability $\alpha = \min \left\{ \frac{p(x^{(i)}) p(x^{(i+1)} | x^{(i)})}{p(x^*) p(x^{(i)} | x^*)}, 1 \right\}$.

otherwise reject the sample, i.e. $x^{(i+1)} = x^{(i)}$

- It can be shown that the distribution of $\{x^{(i)}\}$ as $n \rightarrow \infty$ has the same distribution as $p(x)$.

- overall samples are drawn but

→ initial samples depend on $x^{(0)}$ → ignore the initial samples (burn-in phase).

→ samples correlated w/ neighbouring samples → take every m th sample (thinning)

- We need to decide on the proposal distribution z . If z is symmetric, i.e. $p(x^{(i)} | x^{(i+1)}) = p(x^{(i+1)} | x^{(i)})$,

then the probability of accepting the sample is $\alpha = \min \left\{ \frac{p(x^{(i)})}{p(x^{(i+1)})}, 1 \right\}$. true for $z \sim N(0, I)$

Markov chain Monte Carlo (MCMC).

- A MCMC approach is based on the Metropolis-Hastings algorithm to generate samples from a process w/ a limiting distribution Π .

- consider a proposal function — a transition matrix R , where $r_{j,j} = 0$ and $r_{j,k} > 0$ ($j \neq k$).
(the true transition matrix is not known).

- The MCMC algorithm is as follows:

→ Given the current state x_i , select $\hat{x}_{i+1}$ by sampling from the x_i -th row of R .

→ Accept the sample, i.e. $x_{i+1} = \hat{x}_{i+1}$ w/ probability $\alpha = \min \left\{ \frac{\Pi(\hat{x}_{i+1}) R(x_i, \hat{x}_{i+1})}{\Pi(x_i) R(\hat{x}_{i+1}, x_i)}, 1 \right\}$.

otherwise reject the sample, i.e. $x_{i+1} = x_i$

- The process is equivalent to a transition matrix R' where,

$$r'_{j,k} = r_{j,k} \min \left\{ \frac{\Pi(k) r_{j,k}}{\Pi(j) r_{k,j}}, 1 \right\} \text{ if } j \neq k, \quad r'_{j,j} = 1 - \sum_{k \neq j} r_{j,k} \min \left\{ \frac{\Pi(k) r_{j,k}}{\Pi(j) r_{k,j}}, 1 \right\}$$

- We can show that R' and Π are in detailed balance.

$$\Pi(j) r'_{j,k} = \Pi(j) r_{j,k} \min \left\{ \frac{\Pi(k) r_{j,k}}{\Pi(j) r_{k,j}}, 1 \right\} = \min \{ \Pi(k) r_{j,k}, \Pi(j) r_{k,j} \} = \Pi(k) r_{k,j} \min \left\{ 1, \frac{\Pi(j) r_{j,k}}{\Pi(k) r_{k,j}} \right\} = \Pi(k) r'_{k,j}$$

R' and Π are in detailed balance → Π is a stationary distribution for the transition matrix R' .

By construction, R is irreducible and aperiodic → regular ergodic → Π is the limiting distribution.

Optimisation

Optimisation

The optimisation problem

- The goal of optimisation is to find the independent variables that min./max a given quantity, possibly subject to some restrictions on the allowed range of parameters.
- The quantity to be max/min is the objective / cost / utility / loss function, denoted by $f(x): \mathbb{R}^n \rightarrow \mathbb{R}$.
- The parameters that can be changed are the control / decision variables.
- The restrictions on the allowed parameter values are the constraints.

Mathematically, the optimisation problem is

$$\text{minimise } f(x), x = [x_1, x_2, \dots, x_n]^T \text{ subject to } h_i(x) = 0, i=1, 2, \dots, m \text{ and } g_j(x) \leq 0, j=1, 2, \dots, n$$

where $f(x)$ is the objective function, x is the col. vector of N control variables,

$\{h_i(x), g_j(x)\}$ is the set of constraint functions.

inequality constraints on single variables are bounds, eg. $x_{i,\min} \leq x_i \leq x_{i,\max}$.

+ A min. x of the function $-f(x)$ is a max. x of the function $f(x) \rightarrow$ max. problems can be cast as min. problems.

- Common forms of the objective function $f(x)$ include:

$\rightarrow$ Linear:

$$f(x) = b^T x + c$$

$\rightarrow$ Quadratic:

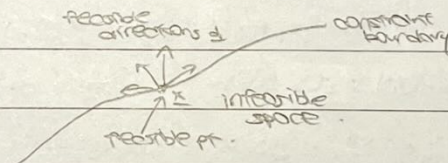
$$f(x) = x^T A x + b^T x + c$$

$\rightarrow$ Non-linear (not linear/quadratic):

$$(eg) f(x) = \exp(x_1) + x^T A x + b^T x + c$$

- When min. $f(x)$ subject to constraints, S is the feasible region, and only $x \in S$ is a feasible sol'n. For an unconstrained problem, S is infinitely large.

- At a feasible pt. x , a direction d is a feasible direction if an arbitrarily small move from x in direction d remains feasible.



Stationary points and types of minima.

- If $f(x)$ is smooth, s.t. $\nabla f(x)$ exists, then x^* is a stationary pt. of f if

$$\nabla f(x^*) = 0$$

+ minima, maxima and saddle pts are stationary pts.

- Types of minima includes the following:

$\rightarrow$ Global min.

$$f(x^*) \leq f(y) \quad \forall y \in S$$

$\rightarrow$ Weak local min.

$$f(x^*) \leq f(y) \quad \forall y = x^* + \epsilon d \in S, y \neq x^*$$

$\rightarrow$ Strong global min.

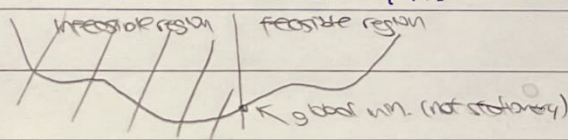
$$f(x^*) < f(y) \quad \forall y \in S, y \neq x^*$$

$\rightarrow$ Strong local min.

$$f(x^*) < f(y) \quad \forall y = x^* + \epsilon d \in S$$

+ Local max/min. are local extrema, and refer to interior local max./min, unless o/w specified.

- When subject to constraints, a global min. might not be a stationary pt.



For Personal Use Only -bkwk2

Necessary and sufficient conditions for a local minimum

- A necessary condition for $\mathbf{x}^*$ to be a local min. of $f(\mathbf{x})$ in S is

$$\nabla f(\mathbf{x}^*) \cdot \mathbf{d} \geq 0 \quad \text{for all feasible directions } \mathbf{d}$$

- If $\mathbf{x}^*$ is an interior pt, i.e. it is not at the boundary of the feasible region, then all dir. are feasible $\rightarrow \mathbf{x}^*$ must be a stationary pt. when $\nabla f(\mathbf{x}^*) = 0$

(the condition $\nabla f(\mathbf{x}^*) = 0$ is necessary but not sufficient for $\mathbf{x}^*$ to be a min. as the above condition holds for max / saddle pts)

- For the univariate case ($x \in \mathbb{R}$), the Taylor expansion about $\mathbf{x}^*$ is

$$f(x) = f(x^*) + (x - x^*)f'(x^*) + \frac{1}{2}(x - x^*)^2 f''(x^*) + \text{H.O.T.}$$

$$f(x) - f(x^*) = (x - x^*)f'(x^*) + \frac{1}{2}(x - x^*)^2 f''(x^*) + \text{H.O.T.}$$

If $\mathbf{x}^*$ is an interior stationary pt, $f'(x^*) = 0$, then we have a strong local min. if

$$f(x) - f(x^*) \approx \frac{1}{2}(x - x^*)^2 f''(x^*) > 0 \rightarrow f''(x^*) > 0$$

- At a stationary pt $\mathbf{x}^*$, $f''(x^*) > 0$ is a sufficient condition of strong local min.

$f''(x^*) \geq 0$ is a necessary condition of strong local min.

(If $f''(x^*) = 0$, we need to analyse the higher order terms)

- For the multivariate case ($\mathbf{x} \in \mathbb{R}^n$), the Taylor expansion about $\mathbf{x}^*$ is

$$f(\mathbf{x}) = f(\mathbf{x}^*) + (\mathbf{x} - \mathbf{x}^*)^T \nabla f(\mathbf{x}^*) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^T \mathbf{H}(\mathbf{x}^*) (\mathbf{x} - \mathbf{x}^*) + \text{H.O.T.}$$

$$f(\mathbf{x}) - f(\mathbf{x}^*) = (\mathbf{x} - \mathbf{x}^*)^T \nabla f(\mathbf{x}^*) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^T \mathbf{H}(\mathbf{x}^*) (\mathbf{x} - \mathbf{x}^*) + \text{H.O.T.}$$

where $\mathbf{H}(\mathbf{x}) = \nabla(\nabla f(\mathbf{x})) = \nabla \left(\left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right]^T \right) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$ is the Hessian the Hessian $\mathbf{H}(\mathbf{x})$ is symmetric by def'n.

If $\mathbf{x}^*$ is an interior stationary pt, $\nabla f(\mathbf{x}^*) = 0$ then we have a strong local min. if

$$f(\mathbf{x}) - f(\mathbf{x}^*) \approx \frac{1}{2} \mathbf{d}^T \mathbf{H}(\mathbf{x}^*) \mathbf{d} > 0 \rightarrow \mathbf{d}^T \mathbf{H}(\mathbf{x}^*) \mathbf{d} > 0, \quad \text{where } \mathbf{d} = \mathbf{x} - \mathbf{x}^*$$

- At a stationary pt $\mathbf{x}^*$, $\mathbf{d}^T \mathbf{H}(\mathbf{x}^*) \mathbf{d} > 0$ (PD Hessian) is a sufficient condition of strong local min.

$\mathbf{d}^T \mathbf{H}(\mathbf{x}^*) \mathbf{d} \geq 0$ (SPD Hessian) is a necessary condition of strong local min.

- If $\mathbf{H}(\mathbf{x})$ is PD everywhere, then $f(\mathbf{x})$ is strictly convex $\rightarrow$ the min. is unique and a global min.

- A matrix is PD iff all its eigenvalues are +ve. For a 2x2 matrix, it is PD if

$$H_{11} > 0$$

and

$$\det(\mathbf{H}) > 0$$

(If $\mathbf{H}(\mathbf{x}^*) = 0$, we need to analyse the higher order terms)

For Personal Use Only -bkwk2

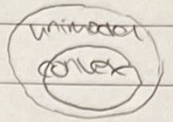
Unimodal and convex functions

- A function is unimodal if it has a single extremum. It is strictly unimodal if the function strictly increases/decreases.

- A function is convex if its graph at any pt x is never below the tangent at any other pt z .

$$f(x) \geq f(z) + \nabla f(z)^T (x-z)$$

(If the function is strictly convex, we replace $\geq$ by $>$)



- convex functions are unimodal but not all unimodal functions are convex

- The local min. of a unimodal/convex function is unique $\rightarrow$ local min. is the global min.

Unconstrained optimization — search methods for unimodal functions

General search algorithm

- The general search algorithm is as follows:

$\hookrightarrow$ 1) start w/ an initial guess x_0 for the min. of $f(x)$, (random or informed)

$\hookrightarrow$ 2) Propose a search dir. d_k

$\hookrightarrow$ 3) propose a step size α_k along d_k (typically by an inner line search loop)

$\hookrightarrow$ 4) Update the estimate of the min location $x_{k+1} = x_k + \alpha_k d_k$

$\hookrightarrow$ 5) Repeat steps 2-5 until convergence

Convergence criteria and rate of convergence

- Convergence can be assessed in diff. ways. For a user-defined tolerance ϵ , we stop the search when

$\hookrightarrow$ Norm of the residual

$$\|f(x_{k+1}) - f(x_k)\| < \epsilon_f$$

$\hookrightarrow$ Norm of the error

$$\|x_{k+1} - x_k\| < \epsilon_x$$

$\hookrightarrow$ Norm of the gradient

$$\|\nabla f(x_k)\| < \epsilon_g$$

$\leftarrow$ used when the curvature is small, and combined w/ test on norm of the error.

- The rate of convergence of an algorithm quantifies how fast the approximate soln approaches the true min. x^* , close to the min, iteration after iteration. Mathematically,

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^p} = \beta$$

where β is the convergence ratio (smaller is better)

and p is the order of convergence (larger is better)

$p=1$: linear convergence
 $p=2$: quadratic convergence

1 The rate of convergence is an asymptotic quantity — it is the speed at which a sequence converges to its limit, if it converges.

For Personal Use Only -bkwk2

Line search

- Line search is the search of a min. for a univariate function. Interval reduction is the best we can do if the gradient $f'(x)$ is not available.

- The line search algorithm is as follows:

↳ 1) Start w/ three pts x_1, x_2, x_3 s.t. $f(x_1) > f(x_3)$ and $f(x_3) < f(x_2)$

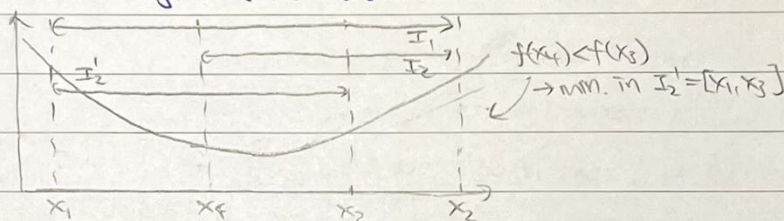
x_1 and x_2 are the bounds and x_3 is the interior pt.

↳ 2) By Bolzano thm, there exists a min. in the interval $I_1 = [x_1, x_2]$ (bracketing)

↳ 3) Compute $f(x_4)$ at a fourth pt $x_4 \in I_1$ for the case $x_4 < x_3$

↳ 4) If $f(x_4) > f(x_3)$, the min. must be in $I_2 = [x_4, x_2]$, otherwise the min. is in $I_2' = [x_1, x_3]$

↳ 5) Repeat steps 2-5 until convergence (interval smaller than tolerance)



Golden section search

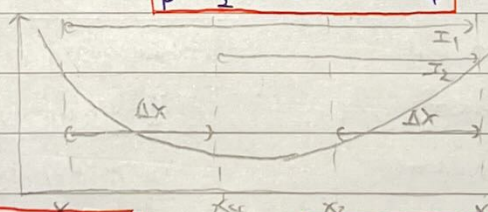
- To find the optimal location of the interior pt x_4 , we impose a const. reduction factor β for the interval (again, w/ linear convergence,

$$\frac{I_2}{I_1} = \frac{I_1 - I_2}{I_2} = \frac{\Delta x}{I_2}$$

setting $\beta = \frac{I_2}{I_1}$ and solving the quadratic yields

$$\beta = \frac{\sqrt{5}-1}{2} \approx 0.618 = \frac{1}{\phi}$$

inverse of the golden ratio



(note that $\Delta x = I_1 - I_2 = I_1(1-\beta)$. Find $\Delta x \rightarrow x_4 = x_1 + \Delta x, x_3 = x_2 - \Delta x$)

- ✓: No need to compute the first or second derivative

✓: Faster convergence than other "non-gradient" methods

x: slower convergence than gradient methods

x: Not robust - goes wrong if the initial interval does not contain a min.

For Personal Use Only -bkwk2

Fitting with a quadratic polynomial at three points

- we can approx. the objective function $f(x)$ w/ a quadratic function $q(x)$ by matching them at three pts.

$$q(x) = a_0 + a_1x + a_2x^2$$

we can solve the min. of $q(x)$ analytically $x^* = -\frac{a_1}{2a_2}$, which approx the min. of $f(x)$.

- to obtain the coefficients, we impose.

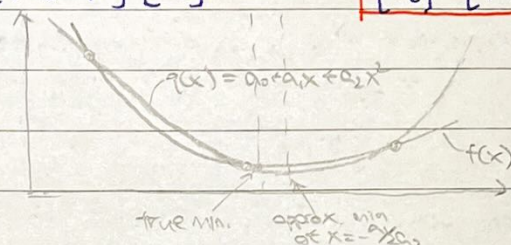
$$f(x_1) = a_0 + a_1x_1 + a_2x_1^2$$

$$f(x_2) = a_0 + a_1x_2 + a_2x_2^2$$

$$f(x_3) = a_0 + a_1x_3 + a_2x_3^2$$

using matrix form and solving, we have

$$\begin{bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} \rightarrow \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix}^{-1} \begin{bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \end{bmatrix}$$



Fitting with a quadratic polynomial at one point - Newton's method

- Instead of fitting the function at three pts, we can fit the function value, first and second derivatives evaluated at a single pt.

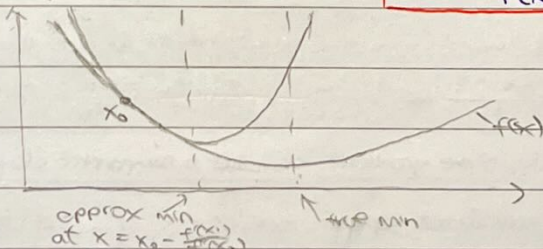
$$f(x) = f(x_0) + (x-x_0)f'(x_0) + \frac{1}{2}(x-x_0)^2f''(x_0) + \text{H.O.T.}$$

We get a quadratic approx. $q(x)$ of $f(x)$ by neglecting the higher order terms,

$$q(x) = f(x_0) + (x-x_0)f'(x_0) + \frac{1}{2}(x-x_0)^2f''(x_0)$$

- to calculate the min. of $q(x)$, we set its derivative to zero,

$$q'(x) = f'(x_0) + (x-x_0)f''(x_0) = 0 \rightarrow x = x_0 - \frac{f'(x_0)}{f''(x_0)}$$



- ✓ : when it works, has quadratic convergence ($p=2$)

✓ : could be generalised to multidimensional searches (but could be computationally expensive)

x : sensitive to the choice of the initial pt $x_0 \rightarrow$ could make the estimate worse or even converge to the wrong extremum (usually when $f'(x)$ is zero or negative)

For Personal Use Only -bkwk2

Quasi-Newton methods

- The computation of the second derivatives can be expensive (especially for higher dim.)

A large no. of methods are designed to approx. the second derivative.

- The simplest approx is the secant method, which takes the current and previous pts. to estimate the second derivative of the current pt.

$$f''(x_1) \approx \frac{f'(x_1) - f'(x_0)}{x_1 - x_0}$$

- The Taylor expansion abt. x_1 is approx. as

$$S(x) = f(x_1) + (x - x_1)f'(x_1) + \frac{1}{2}(x - x_1)^2 \frac{f'(x_1) - f'(x_0)}{x_1 - x_0}$$

Setting $S'(x) = 0$ yields the estimate for the min.

$$S'(x) = f'(x_1) + (x - x_1) \frac{f'(x_1) - f'(x_0)}{x_1 - x_0} = 0 \rightarrow x = x_1 - f'(x_1) \frac{x_1 - x_0}{f'(x_1) - f'(x_0)}$$

- convergence for the secant method is superlinear ($p = \frac{\log 2}{2} \approx 1.618 > 1$)

- Quasi-Newton methods have higher dim. generalizations: Broyden's method, Broyden-C-G-Broyden method.

Unconstrained optimisation — search methods for multivariate functions

steepest-descent method

- Access to the gradient enables gradient search — more efficient than direct (gradient-free) search

- we can expect the fact that the gradient is the dir. of max first-order change of a multivariate function $f(x)$. The steepest-descent algorithm is as follows:

1) start w/ an initial guess for the min. x_0 .

2) set the search dir. as the -ve gradient,

$$d_k = -\nabla f(x_k)$$

main loop changes the direction

3) determine by line search the step size α_k that min. $f(x)$ along d_k .

4) update the sol'n to $x_{k+1} = x_k + \alpha_k d_k$

inner loop is optimisation in 1D

5) repeat steps 2-4 until convergence.

- Ideally, the line search in step 3 should satisfy $\frac{df(x_k + \alpha_k d_k)}{d\alpha_k} = 0$, which implies that the gradient at $x_{k+1} = x_k + \alpha_k d_k$ is orthogonal to d_k , i.e. successive search dir. are orthogonal $d_k^T d_{k+1} = 0$

$$0 = \frac{df(x_k + \alpha_k d_k)}{d\alpha_k} = \frac{df(x_{k+1})}{d\alpha_k} = \frac{\partial f(x_{k+1})}{\partial x_{k+1}} \frac{\partial x_{k+1}}{\partial \alpha_k} = \nabla f(x_{k+1}) \cdot \frac{\partial (x_k + \alpha_k d_k)}{\partial \alpha_k} = (-d_k) \cdot d_k$$

- Practically, an approximate line search is performed, so $\frac{df(x_k + \alpha_k d_k)}{d\alpha_k} \approx 0 \rightarrow d_k^T d_{k+1} \approx 0$

approx too poor $\rightarrow$ they not converge
approx too good $\rightarrow$ waste resources

- If we approximate the objective function $f(x)$ w/ a 2nd order Taylor expansion, we can get a

$$\text{closed form optimum step size } \alpha_k, \quad \alpha_k = -\frac{\nabla f(x_k)^T d_k}{d_k^T H(x_k) d_k}$$

$$f(x_k + \alpha_k d_k) \approx f(x_k) + \nabla f(x_k)^T (\alpha_k d_k) + \frac{1}{2} (\alpha_k d_k)^T H(x_k) (\alpha_k d_k) + \text{H.O.T.}$$

$$\frac{df(x_k + \alpha_k d_k)}{d\alpha_k} = \nabla f(x_k)^T d_k + \alpha_k d_k^T H(x_k) d_k = 0 \rightarrow \alpha_k = -\frac{\nabla f(x_k)^T d_k}{d_k^T H(x_k) d_k}$$

- However, if we have the Hessian $H(x_k)$, better search methods (e.g. Newton-Raphson) are available.

For Personal Use Only -bkwk2

convergence of steepest-descent method

- the steepest-descent algorithm converges linearly, i.e. $p=1$

$$\|x_{k+1} - x^*\| = \beta \|x_k - x^*\|$$

β is close to 1 $\rightarrow$ steepest-descent is slow.

- It can be shown using Kantorovich inequality that

$$\beta \leq \left(\frac{k+1}{k-1} \right)^2 \quad \leftarrow \begin{array}{l} k \rightarrow 1 \rightarrow \beta \rightarrow 0 \text{ (good)} \\ k \rightarrow \infty \rightarrow \beta \rightarrow 1 \text{ (bad)} \end{array}$$

where k is the ℓ_2 condition no. of $H(x^*)$, $k = \frac{\lambda_{\max}(H(x^*))}{\lambda_{\min}(H(x^*))}$ $\leftarrow$ note that the Hessian $H(x^*)$ is hermitian (symmetric).

- in typical engineering problems, $H(x^*)$ is quite ill-conditioned $\rightarrow \beta$ is close to 1.

- ill-conditioned quadratic functions have elongated valleys or isocontours $\rightarrow$ long valley of small gradients and a large distance to move $\rightarrow$ slow to converge.

Newton-Raphson method.

- Assume we can evaluate the gradient $\nabla f(x)$ and the Hessian $H(x)$.

- $f(x)$ is approx. by a quadratic function using the function value, gradient and Hessian at the current location.

$$f(x) \approx f(x_k) + \nabla f(x_k)^T (x - x_k) + \frac{1}{2} (x - x_k)^T H(x_k) (x - x_k) + \text{H.O.T.}$$

we get a quadratic approx $q(x)$ of $f(x)$ by neglecting the higher order terms,

$$q(x) = f(x_k) + \nabla f(x_k)^T (x - x_k) + \frac{1}{2} (x - x_k)^T H(x_k) (x - x_k)$$

- To calculate the min. of $q(x)$, we set its gradient to zero,

$$\nabla q(x) = \nabla f(x_k) + H(x_k)(x - x_k) = 0 \quad \rightarrow \quad x = x_k - [H(x_k)]^{-1} \nabla f(x_k)$$

- we use the min. of $q(x)$ as our next guess for the min. of $f(x)$.

$$x_{k+1} = x_k + \alpha_k g_k, \quad \alpha_k = 1 \text{ and } g_k = -[H(x_k)]^{-1} \nabla f(x_k)$$

- we can extend the Newton-Raphson method by finding the step size α_k using line search at each iteration $\rightarrow$ speeds up convergence, will find a min. (not max/saddle).

convergence of Newton-Raphson method.

- the Newton-Raphson algorithm converges quadratically, i.e. $p=2$

$$\|x_{k+1} - x^*\| \leq \beta \|x_k - x^*\|^2$$

It converges to the min. in 1 iteration in a quadratic function $f(x)$.

- Newton-Raphson is not very robust $\rightarrow$ it may take the iterations away from a min. / not converge if the initial guess is not appropriate.

- w/o line search for step size α_k , if $H(x)$ is not PD, the method may converge to max./saddle.

- Newton-Raphson requires computing + inverting the Hessian, computation $\sim O(n^2)$, inversion $\sim O(n^3)$

Barzilai-Borwein method

- We can generalise the secant method for higher dimensions. The Hessian $H(x_k)$ in Newton-Raphson is approx as follows in the Barzilai-Borwein method.

Quasi-Newton method

$$x = x_1 - \nabla f(x_k) \frac{(x_1 - x_0)^T (\nabla f(x_1) - \nabla f(x_0))}{\|\nabla f(x_1) - \nabla f(x_0)\|^2}$$

- Barzilai-Borwein method only requires computing the gradient $\nabla f(x_k)$, and is usually more stable than Newton's method or other Quasi-Newton methods.
- Not much is known about its convergence.

Conjugate gradient method.

- If we could step along the eigendirections, the convergence would be faster than steepest-descent. (eigendirections are good search dir. as they represent the principal axes of ellipsoids)

- However, computing the eigenvectors $\sim O(N^3)$ - as bad as Hessian-based method $\rightarrow$ don't use eigenvectors.
- Consider a set of A -conjugate vectors $\{d_1, \dots, d_n\}$, i.e.

true for symmetric A .

$$d_i^T A d_j = 0 \quad \text{for } i \neq j.$$

(Orthogonal eigenvectors fulfill the conjugacy condition, but there are other vectors that satisfy this)

- Consider transforming the coords $x \rightarrow y$, where $x = \underline{S}y$, $\underline{S} = \begin{bmatrix} \uparrow d_1 & \uparrow d_2 & \dots & \uparrow d_n \\ \downarrow & \downarrow & \dots & \downarrow \end{bmatrix}$.

For a quadratic function $f(x) = \frac{1}{2} x^T A x - b^T x$, we have

$$f(x) = f(\underline{S}y) = \frac{1}{2} (\underline{S}y)^T A (\underline{S}y) - b^T (\underline{S}y) = \frac{1}{2} y^T (\underline{S}^T A \underline{S}) y - (\underline{S}^T b)^T y.$$

By construction, $\underline{S}^T A \underline{S} = \underline{\Lambda}$.

$$\underline{S}^T A \underline{S} = \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ d_1 & d_2 & \dots & d_n \\ \downarrow & \downarrow & \dots & \downarrow \end{bmatrix}^T A \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ d_1 & d_2 & \dots & d_n \\ \downarrow & \downarrow & \dots & \downarrow \end{bmatrix} = \begin{bmatrix} d_1^T A d_1 & d_1^T A d_2 & \dots & d_1^T A d_n \\ d_2^T A d_1 & d_2^T A d_2 & \dots & d_2^T A d_n \\ \vdots & \vdots & \ddots & \vdots \\ d_n^T A d_1 & d_n^T A d_2 & \dots & d_n^T A d_n \end{bmatrix} = \begin{bmatrix} d_1^T A d_1 & 0 & \dots & 0 \\ 0 & d_2^T A d_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & d_n^T A d_n \end{bmatrix} = \underline{\Lambda}$$

 $\rightarrow$ The problem in the y space is a quadratic form along the axes (search dir. $\{d_1, \dots, d_n\}$).

- At iteration k , we have the gradient $\nabla f(x_k)$ and the previous search dir. $\{d_1, \dots, d_{k-1}\}$.

We construct the new search dir. as

$$d_k = -\nabla f(x_k) + \beta_k d_{k-1}$$

β_k has memory of the previous dir. d_{k-1} .

Imposing the conjugacy condition $d_{k-1}^T A d_k = 0$,

$$d_{k-1}^T A (-\nabla f(x_k) + \beta_k d_{k-1}) = -d_{k-1}^T A \nabla f(x_k) + \beta_k d_{k-1}^T A d_{k-1} \rightarrow \beta_k = \frac{d_{k-1}^T A \nabla f(x_k)}{d_{k-1}^T A d_{k-1}}$$

It can be shown that w/ this choice of β_k , the search dir. d_k is A -conjugate to all previous search dir.

- The conjugate gradient algorithm is as follows:

1) Initialise w/ steepest descent $d_0 = -\nabla f(x_0)$ 2) Update location and determine d_k w/ line search, $x_{k+1} = x_k + \alpha_k d_k$ 3) Find a new search dir d_{k+1} , $d_{k+1} = -\nabla f(x_{k+1}) + \beta_{k+1} d_k = -\nabla f(x_{k+1}) + \left[\frac{\|\nabla f(x_{k+1})\|^2}{\|\nabla f(x_k)\|^2} \right] d_k$

4) Repeat steps 2-3 until convergence

For Personal Use Only -bkwk2

Convergence of Conjugate gradient method.

- The conjugate gradient algorithm converges linearly, i.e. $p=2$

$$\|x_{k+1} - x^*\| = \beta \|x_k - x^*\|$$

β is close to 0 $\rightarrow$ conjugate gradient is superlinear.

- For large ill-conditioned problems, it can be shown that

$$\beta \approx \left(1 - \frac{2}{\kappa}\right)$$

where κ is the L_2 condition no.

- The conjugate gradient method does not req. the Hessian $H(x^*)$ and its inversion $(H(x^*))^{-1}$

- The conjugate gradient method is a Krylov subspace method - for a N -dimensional objective function $f(x)$, it converges after N iterations.

- Numerical errors can lead to loss of conjugacy $\rightarrow$ periodically reset by setting $g_k = -\nabla f(x_k)$.

- The conjugate gradient method is originally used to solve linear systems $Ax = b$.

For optimisation of quadratic function, $f(x) = \frac{1}{2}x^T Ax - b^T x \rightarrow \nabla f(x) = Ax - b = 0$

Summary of multi-dimensional search-direction methods.

Method.	Advantages	Disadvantages
Steepest-descent	Only needs gradient	Linear convergence slow for ill-conditioned problems
Newton-Robinson	Quadratic convergence (more reliable w/ line search)	Needs Hessian can diverge
Bartolai-Borwein	Very stable Faster than steepest descent	convergence analysis difficult
conjugate gradient	Faster than steepest descent	Linear convergence

Least squares fitting of a model to data.

- Let $y^{(j)}$ be the measured data, and $\phi_j(x)$ be the model's prediction for $y^{(j)}$. $j=1, 2, \dots, m$

$x = [x_1, \dots, x_n]^T$ is the vector of independent parameters, which can be tuned to fit the model to data.

- The elements of the residual vector $r(x)$ are the errors in the model's prediction for each data point.

$$r_j(x) = \phi_j(x) - y^{(j)}$$

The cost function we want to min. is the sum of squared errors.

$$f(x) = \sum_{j=1}^m r_j(x)^2 = r(x)^T r(x)$$

The gradient $\nabla f(x)$ is given by

$$\frac{\partial f}{\partial x_i} = \frac{\partial}{\partial x_i} \left(\sum_{j=1}^m r_j(x)^2 \right) = 2 \sum_{j=1}^m \frac{\partial r_j(x)}{\partial x_i} r_j(x) \rightarrow \nabla f(x) = 2J(x)^T r(x)$$

where $J(x) = \begin{bmatrix} \frac{\partial \phi_1}{\partial x_1} & \dots & \frac{\partial \phi_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial \phi_m}{\partial x_1} & \dots & \frac{\partial \phi_m}{\partial x_n} \end{bmatrix}$ is the Jacobian matrix of $r(x)$.

The Hessian $H(x)$ is given by

$$\frac{\partial^2 f}{\partial x_i^2} = \frac{\partial}{\partial x_i} \left(2 \sum_{j=1}^m \frac{\partial r_j(x)}{\partial x_i} r_j(x) \right) = 2 \sum_{j=1}^m \frac{\partial^2 r_j(x)}{\partial x_i^2} r_j(x) + 2 \sum_{j=1}^m \frac{\partial r_j(x)}{\partial x_i} \frac{\partial r_j(x)}{\partial x_i} \rightarrow H(x) = 2 \sum_{j=1}^m r_j(x) P_j(x) + 2J(x)^T J(x)$$

where $P_j(x) = \nabla(\nabla f_j(x))$ is the Hessian of the residual $r_j(x)$

For Personal Use Only -bkwk2

Gauss-Newton method

- The Gauss-Newton method assumes that the residuals at the optimum $r_j(x^*)$ are small, therefore $R_j(x)$ can be neglected, so the Hessian $H(x)$ can be approximated as:

$$H(x) \approx \tilde{H}(x) = 2J(x)^T J(x)$$

for x near the minimum x^* , and $\tilde{H}(x)$ is PD.

- Applying the approx. for the Hessian $H(x)$ to the Newton-Raphson method, the search dir. Δx is

$$\Delta x = -[\tilde{H}(x_k)]^{-1} \nabla f(x_k) = -[2J(x_k)^T J(x_k)]^{-1} 2J(x_k)^T r(x_k) = -J(x_k)^+ r(x_k)$$

where $J(x_k)^+ = [J(x_k)^T J(x_k)]^{-1} J(x_k)^T$ is the pseudo inverse matrix.
 cannot distribute the -1 as $J(x_k)$ may not be square.

- The estimate is updated as for the Newton-Raphson method, $x_{k+1} = x_k + \Delta x$.
- The Gauss-Newton method only requires the Jacobian (first derivatives) and avoids computing or inverting the Hessian $H(x)$.
- The Gauss-Newton method converges nearly quadratically if the initial residuals are small (o/w, it has poor performance)

Constrained optimisation — Linear programming.

Linear Programming (LP)

- LP is an optimisation problem in which both the objective and constraints are linear
- The standard / canonical form of a linear program is as follows:

$$\begin{aligned} &\text{Minimise} && f(x) = c^T x, \quad x \in \mathbb{R}^n \\ &\text{subject to } m \text{ equality constraints} && Ax = b, \quad A \in \mathbb{R}^{m \times n} \\ &\text{and bounds} && x_i \geq 0, \quad i = 1, \dots, n \end{aligned}$$

* We assume that $\text{rank}(A) = m$

- If there are inequality constraints of the form $a_j^T x \geq b_j$, we can convert them into equality constraints by introducing a new slack variable s_j , $s_j = a_j^T x - b_j$, where $s_j \geq 0$.
- The m constraints $Ax = b$ define a subspace of $\mathbb{R}^n$ (usually $m < n$ o/w the feasible set is empty / $A^T b$)
- The bounds $x_i \geq 0$ cut out a portion of this subspace to define the feasible region (convex polyhedron)
- The feasible region has a no. of corners — extremal pts, which are the basic solns. The basic solns have m non-zero basic variables and $n-m$ zero free (non-basic) variables
- The isocontours of f are a series of parallel $(n-1)$ -dim hyperplanes. As f increases, the first isocontour to intersect the feasible region will do so at the minimum.
- * We assume the problem is non-degenerate $\rightarrow$ there are m (non-zero) basic variables (o/w, cycling may occur)
- * For low dimensional problems, LP can be solved graphically.

For Personal Use Only -bkwk2

fundamental theorem of linear programming.

- Given a LP in standard form where $A \in \mathbb{R}^{m \times n}$ and $\text{rank}(A) = m$, if there is a feasible sol'n, there is a basic feasible sol'n; if there is an optimal feasible sol'n, there is an optimal basic feasible sol'n.

→ We can solve a linear program by inspecting basic feasible sol'ns until we find the optimum.

the simplex algorithm.

- Large LPs cannot be easily solved by hand → simplex algorithm systematically solves LPs.

- The simplex algorithm makes two phases:

(i) Phase 1: start the algorithm.

↳ Find an initial basic feasible sol'n if there is not an obvious one

↳ start from the initial basic feasible sol'n (vertex of feasible region w/ m (not-zero) basic variables)

(ii) Phase 2: Move from one vertex to another until the min. is found

↳ Move along an edge of the feasible region to another vertex where f is smaller

↳ Repeat until you find a vertex where edge leading away would increase f → this is the min.

Phase 2: Move along one vertex to another (tableau method)

- Step 0: Formalise the problem

↳ minimise $c^T x = 200x_0 + 400x_1 + 680x_2 + 800x_3$

subject to $Ax = b \Leftrightarrow \begin{bmatrix} 3 & 7 & 5 & 3 \\ 0 & 3 & 5 & 7 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 80 \\ 40 \end{bmatrix} \Leftrightarrow \begin{cases} 3x_0 + 7x_1 + 5x_2 + 3x_3 = 80 \\ 3x_1 + 5x_2 + 7x_3 = 40 \end{cases}$

and $x_i \geq 0$

↳ We can collect the constraint eqns and the objective function coeff into a single tableau (mtrix) (mtrix)

$$\begin{bmatrix} A & b \\ c^T & 0 \end{bmatrix} = \begin{bmatrix} 3 & 7 & 5 & 3 & 80 \\ 0 & 3 & 5 & 7 & 40 \\ 200 & 400 & 680 & 800 & 0 \end{bmatrix} \begin{matrix} \text{constraints} \\ \text{cost function} \end{matrix}$$

- Step 1: start from a basic sol'n.

↳ From phase 1, we get an initial basic feasible sol'n $\begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 10 \\ 2 \\ 0 \end{bmatrix} \rightarrow x_1, x_2$ are the basic variables.

↳ Manipulate the tableau to get canonical form. (under basic variables, we get $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$)

Eliminate the coeff. of the basic variables in the last line (get reduced cost, -ve of objective function)

$$\begin{aligned} [4] &: \frac{1}{4} \times ([1] - [2]) \\ [5] &: \frac{1}{5} \times ([2] - 3 \times [4]) \\ [6] &: [3] - 400[4] - 680[5] \end{aligned} \quad \begin{bmatrix} x_0 & x_1 & x_2 & x_3 & \text{RHS} \\ 3/4 & 1 & 0 & -1 & 10 \\ -9/20 & 0 & 1 & 2 & 2 \\ 206 & 0 & 0 & -160 & -5360 \end{bmatrix} \begin{matrix} [4] \\ [5] \\ [6] \end{matrix}$$

- Step 2: Identify the edge to follow

↳ choose the entering variable to be the one w/ the most -ve reduced cost → x_3 is EV

↳ To choose the leaving variable, consider the ratios $\frac{\text{RHS}}{x_3}$ for each row, i.e. $\frac{10}{-1}$ vs $\frac{2}{2}$

We choose the variable corresponding to the smallest, -ve ratio → x_2 is LV

- Step 3: calculate the next vertex

↳ Repeat steps 1, 2 until all reduced costs are +ve.

$$\begin{bmatrix} 0.525 & 1 & 0.5 & 0 & 11 \\ -0.225 & 0 & 0.5 & 1 & 1 \\ 170 & 0 & 80 & 0 & 5200 \end{bmatrix} \rightarrow \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 10 \\ 1 \\ 1 \end{bmatrix}$$

For Personal Use Only -bkwk2

Phase 1: start the algorithm

- To find an initial basic feasible sol'n, we construct an auxiliary optimisation problem, where the objective function is the "error" that measures by how much the constraints are violated.
- The optimal sol'n of this problem has zero error $\rightarrow$ basic feasible sol'n.
- We create m slack variables, add one to each of the constraint eqns. The objective function is the sum of the slack variables. (the slack variables are the initial basic variables).
- consider the previous example. The auxiliary problem is

$$\begin{aligned} 3x_0 + 7x_1 + 5x_2 + 3x_3 + s_1 + 0 &= 80 \\ 0 + 3x_1 + 5x_2 + 7x_3 + 0 + s_2 &= 40 \end{aligned}$$

The auxiliary objective function is $s_1 + s_2$, so the auxiliary tableau is .

$$\begin{array}{cccccc|c} x_0 & x_1 & x_2 & x_3 & s_1 & s_2 & RHS \\ \hline 3 & 7 & 5 & 3 & 1 & 0 & 80 \\ 0 & 3 & 5 & 7 & 0 & 1 & 40 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{array} \begin{array}{l} [1] \\ [2] \\ [3] \end{array}$$

Eliminating the coeff of the basic variables in the last line,

$$\begin{array}{cccccc|c} x_0 & x_1 & x_2 & x_3 & s_1 & s_2 & RHS \\ \hline 3 & 7 & 5 & 3 & 1 & 0 & 80 \\ 0 & 3 & 5 & 7 & 0 & 1 & 40 \\ -3 & -10 & -10 & -10 & 0 & 0 & -120 \end{array} \begin{array}{l} [1] \\ [2] \\ [4] \end{array}$$

$[4] = [3] - [1] \cdot [2]$

Then solve the auxiliary tableau as in phase 2.

- After solving the auxiliary tableau, if $s_1 = s_2 = 0$ (slack variables are non-basic variables), and the obj function equals 0, then we have found the initial basic feasible sol'n.
- If $s_1 > 0$ and $s_2 > 0$, there is no feasible sol'n to the original problem.

Constrained optimisation — non-linear constrained optimisation

Nonlinear constrained optimisation

- consider the optimisation problem

$$\begin{aligned} &\text{minimise} && f(x) && x \in \mathbb{R}^N \\ &\text{subject to} && h_i(x) = 0 && i=1,2,\dots,m \quad [\text{equality constraints}] \\ &&& g_j(x) \leq 0 && j=1,2,\dots,n \quad [\text{inequality constraints}] \end{aligned}$$

generally non-linear

The set of equality and inequality constraints defines the feasible region.

- At a location x , constraints are
 - $\hookrightarrow$ active if they do not allow x to infinitesimally change in at least one dir. *equality, active inequalities*
 - $\hookrightarrow$ inactive if they allow x to infinitesimally change in any dir. *inactive inequalities*
- In N dimensions, it is not possible to have a sol'n w/ more than N independent constraints active.

For Personal Use Only -bkwk2

Optimality condition on active constraints

- A necessary condition for x^* to be a local min. is

$$\nabla f(x^*) \cdot d \geq 0 \quad \text{for all feasible directions } d.$$

In constrained optimization, some direction d may not be feasible, so in general,

$$\nabla f(x^*) \cdot d \geq 0 \not\Rightarrow \nabla f(x^*) = 0$$

- consider that x is subject to one equality constraint $h(x) = 0$.

Let d be the tangent dir. to the constraint. Suppose $\pm d$ is a feasible dir.

$\hookrightarrow$ If $\nabla f(x^*) \cdot d < 0 \rightarrow x$ is not a min.

$\hookrightarrow$ If $\nabla f(x^*) \cdot d > 0 \rightarrow$ in opposite dir $d' = -d$, $\nabla f(x^*) \cdot d' < 0 \rightarrow x$ is not a min.

$\Rightarrow x$ is a min. if $\nabla f(x^*) \cdot d = 0$.

Geometrically, $\nabla f(x^*) \perp d \rightarrow \nabla f(x^*) \parallel \nabla h(x^*)$, i.e. $\nabla f(x^*) = -\lambda \nabla h(x^*)$ where λ is a scalar

- w/ multiple independent active constraints, at the min, ∇f must be perpendicular to the tangent plane of the constraint intersection, which has dimensionality $n-m$. The optimality condition is thus

$$\nabla f(x^*) = -\sum_{i=1}^m \lambda_i \nabla h_i(x^*) = -\nabla h(x^*)^T \underline{\lambda}, \quad \text{where } \lambda_i, i=1,2,\dots,m \text{ are scalars}$$

Lagrangean and optimality condition for equality constraints

- We can recover the optimality condition by defining an unconstrained min. problem w/ a new cost function

$$L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i h_i(x) \quad \text{note } h_i(x) = 0$$

where $L(x, \lambda)$ is the Lagrangean and λ_i are the Lagrangean multipliers (scalars)

- x^* is a stationary pt. of the Lagrangean $L(x, \lambda)$ if

$$\nabla L(x^*, \lambda) = 0 \Rightarrow \nabla f(x^*) + [\nabla h(x^*)]^T \underline{\lambda} = 0 \quad \text{and} \quad h_i(x^*) = 0 \quad \text{for } i=1,2,\dots,m$$

A sufficient condition for x^* to be a minimum is

$$d^T \nabla^2 L(x^*, \lambda) d > 0 \quad \text{where } d \in \text{space tangent to all constraints.}$$

i.e. the Hessian $\nabla^2 L(x^*, \lambda)$ is PD in the tangent space.

- + If the Lagrangean multiplier is zero, $\lambda_i = 0$, then $\nabla f(x^*) = 0$ at that pt.

For Personal Use Only -bkwk2

Lagrangian and optimality conditions for inequality constraints

- For inequality constraints, we define the Lagrangian

$$L(x, M) = f(x) + \sum_{j=1}^n M_j g_j(x) \quad \text{Note } g_j(x) \leq 0$$

- x^* is a stationary pt. of the Lagrangian $L(x, M_j)$ if

$$\nabla L(x^*, M_j) = 0 \Rightarrow \nabla f(x^*) + [\nabla g_j(x^*)]^T M_j = 0 \quad \text{and} \quad M_j g_j(x^*) = 0, \quad M_j \geq 0 \quad \text{for } j=1, 2, \dots, n$$

These are the Karush-Kuhn-Tucker (KKT) conditions, M_j are the KKT multipliers.

- For active constraints, $g_j(x) = 0$ [constraint becomes an equality constraint]

For inactive constraints, $g_j(x) < 0$ and $M_j = 0$

- In the general case w/ both equalities and inequality constraints,

$$L(x, \lambda, M_j) = f(x) + \sum_{i=1}^m \lambda_i h_i(x) + \sum_{j=1}^n M_j g_j(x)$$

Sensitivity,

- Consider perturbing the sol'n x^* by an infinitesimally small amount, i.e. $x^* \rightarrow x^* + \delta x$.

$$\nabla f(x^*) + [\nabla h(x^*)]^T \lambda + [\nabla g(x^*)]^T M = 0 \rightarrow \delta x^T \nabla f(x^*) = -\delta x^T ([\nabla h(x^*)]^T \lambda + [\nabla g(x^*)]^T M)$$

$$\therefore \delta f(x^*) = -\delta h(x^*)^T \lambda - \delta g(x^*)^T M$$

$\rightarrow$ change in cost function due to a change in the sol'n is prop. to the -ve Lagrangian / KKT multipliers.

i.e. $-\lambda_i$ and M_j are the sensitivities of f to an infinitesimal change in the constraint h_i and g_j respectively.

Penalty functions and barrier functions

- w/ penalty functions, we do not enforce the constraints exactly, but only approximate controllably.

(i.e. soft constraints), we define a new unconstrained problem

$$q(x, K) = f(x) + K \sum_{i=1}^m h_i(x)^2$$

where K is the penalty parameter and $K \sum_{i=1}^m h_i(x)^2$ is the penalty function.

- It can handle both equalities and inequalities (for inequalities, we $K \sum_{j=1}^n \max[0, g_j(x)]^2$).

- For large K , $h(x)$ needs to be very small $\rightarrow$ problem close to constrained problem. For small K , $h(x)$ does not need to be small $\rightarrow$ constraint violated more, usually start w/ small K and increase it.

- w/ barrier functions, the feasibility is enforced exactly by using a penalty that diverges as we approach the boundary of the constraint. We define a new unconstrained problem

inverse barrier function

$$q(x, K) = f(x) - \frac{1}{K} \sum_{j=1}^n \frac{1}{g_j(x)}$$

logarithmic barrier function

$$q(x, K) = f(x) - \frac{1}{K} \sum_{j=1}^n \ln[-g_j(x)]$$

- It can only handle inequalities and always yields a feasible answer.

- It req. a feasible starting pt and cannot iterate through infeasible space.

- It basically restricts the search to the feasible region by penalizing sol'ns near the edge.

- The larger the K , the sharper the divergence of the singularity near the constraint boundary.